UNIVERSIDAD TECNOLÓGICA DE PANAMÁ

FACULTAD DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

DEPARTAMENTO DE COMPUTACIÓN Y SIMULACIÓN DE SISTEMAS

LICENCIATURA EN INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

Asignatura:

Sistemas Operativos

Asignación:

Tarea #1: Identificación y Clasificación de Sistemas Operativos

Profesor:

Prof. Isacio Tamayo

Estudiante:

James Tack

Grupo:

1IL143

Fecha de entrega:

31 de agosto de 2026

2do Semestre 2026

Nota previa sobre las condiciones del laboratorio

Dos cosas cambiaron respecto a lo que plantea la guía, y prefiero aclararlas de entrada para que las capturas se lean sin confusión.

El equipo

Hice el laboratorio en mi computadora personal, no en una de las del aula 3-401. Eso significó que VirtualBox no estaba instalado y tuve que instalarlo antes de la Parte B. La versión que aparece en todas las capturas es la 7.2.16 r174877. El otro detalle es que mi Windows está en inglés, así que los cuadros de diálogo y las salidas de los comandos salen en ese idioma; al lado de cada captura escribo en español el nombre del campo del que se trata.

El archivo .vdi

La guía dice que el docente entrega un .vdi con Linux ya instalado. No lo tuve. La misma guía prevé el caso en su nota final y sugiere usar WSL, pero esa salida obliga a saltarse los pasos 7, 8 y 9, que son justamente donde está la mayor parte del puntaje de la Parte B.

Opté por armar el escenario completo. Descargué la imagen oficial de disco de Ubuntu Server 24.04 LTS del repositorio de Canonical (cloud-images.ubuntu.com), la convertí al formato VDI con VBoxManage y trabajé con ese archivo como si me lo hubieran entregado:

```
REM 1) Conversion de la imagen oficial de Ubuntu al formato .vdi
VBoxManage clonemedium disk "ubuntu-24.04-server-cloudimg-amd64.vmdk" ^
                            "Lab1-Linux-DISCO-DOCENTE.vdi" --format VDI

REM 2) Capacidad del disco virtual a 20 GB
VBoxManage modifymedium disk "Lab1-Linux-DISCO-DOCENTE.vdi" --resize 20480
```

El archivo que sale de ahí se llama Lab1-Linux-DISCO-DOCENTE.vdi y pesa 20,00 GB. Es el que selecciono en el paso 7 con la opción «Usar un archivo de disco duro virtual existente». El usuario y la contraseña de la VM (estudiante / sistemas2026) los aprovisioné con cloud-init, que es el mecanismo normal de estas imágenes y cumple el papel de las credenciales que se entregan junto con el disco.

Los catorce pasos se ejecutaron de verdad en esta máquina. Ninguna captura está montada.

Parte A. Windows por interfaz gráfica y por comandos

Esta parte tiene dos mitades que hacen lo mismo por caminos distintos. Primero miré el sistema como lo ve cualquier usuario, con ventanas y clics. Después repetí las mismas observaciones escribiendo comandos. La idea era comprobar que las dos interfaces le están preguntando al mismo sistema por debajo, aunque presenten las respuestas de forma muy distinta.

Paso 1. Identificar el sistema con winver

Presioné Windows + R, escribí winver y di Enter. Se abre una ventana pequeña con la identificación comercial del sistema.



Figura 1. Ventana «Acerca de Windows» abierta con winver.

Lo que registré:

Campo	Valor
Edición	Windows 11 Pro
Versión	25H2
Compilación (build)	26200.9168
Fabricante	Microsoft Corporation
Licencia a nombre de	Lost

La verificación del paso pedía un número de versión y uno de compilación. Ahí están: 25H2 y 26200.9168.

Paso 2. Contar procesos desde el Administrador de tareas

Abrí el Administrador de tareas con Ctrl + Shift + Esc. La pestaña Procesos de Windows 11 no da un total directo: reparte todo en tres bloques y pone la cantidad entre paréntesis al lado del nombre de cada bloque.

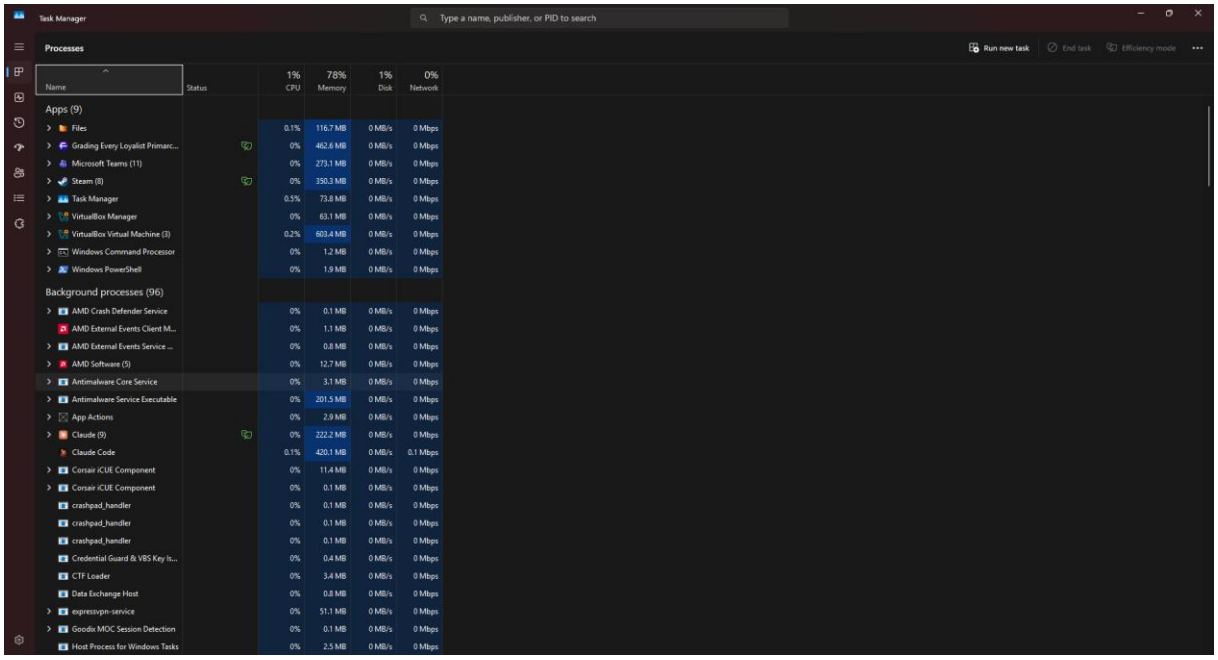


Figura 2. Pestaña Procesos, parte de arriba: Aplicaciones (9) y Procesos en segundo plano (96).

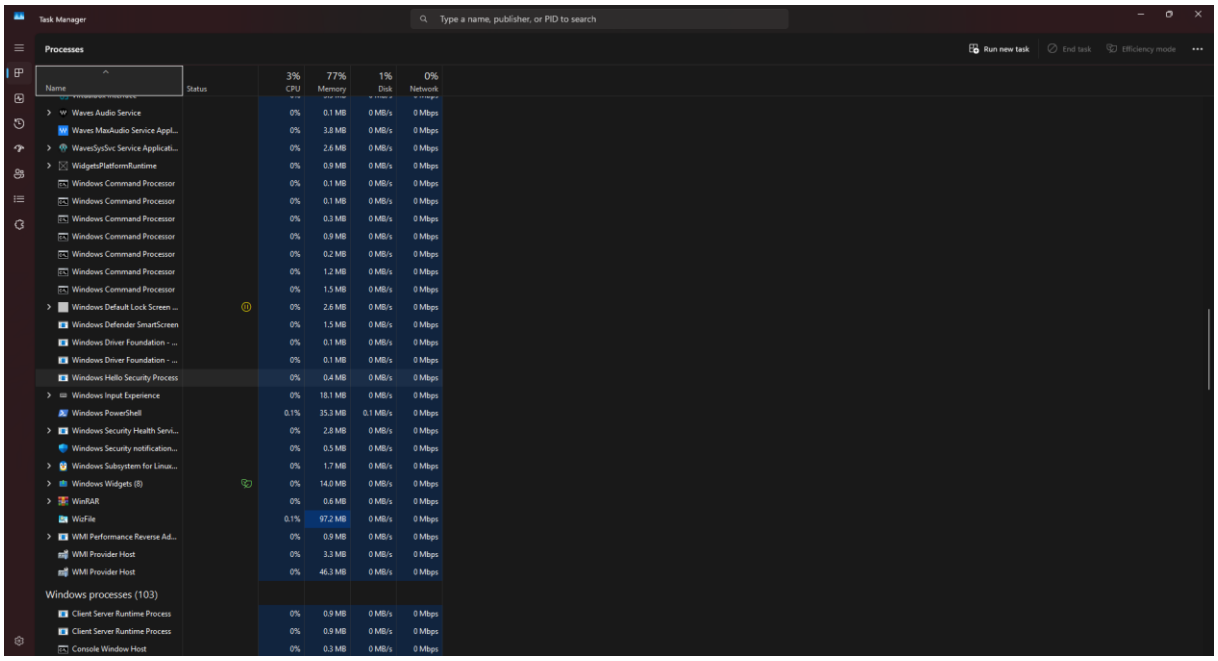


Figura 3. La misma lista más abajo, donde aparece el tercer bloque: Procesos de Windows (103).

Bloque	Cantidad

Bloque	Cantidad
Aplicaciones (Apps)	9
Procesos en segundo plano (Background processes)	96
Procesos de Windows (Windows processes)	103
Suma de las entradas visibles	208

Hay un detalle acá que después me sirvió para responder la primera pregunta de reflexión. Una fila de esa pestaña no siempre es un proceso. Cuando dice «Microsoft Teams (11)», eso es una sola línea en pantalla pero once procesos corriendo. Por eso revisé también la pestaña Detalles, que sí pone una fila por proceso y no agrupa nada.

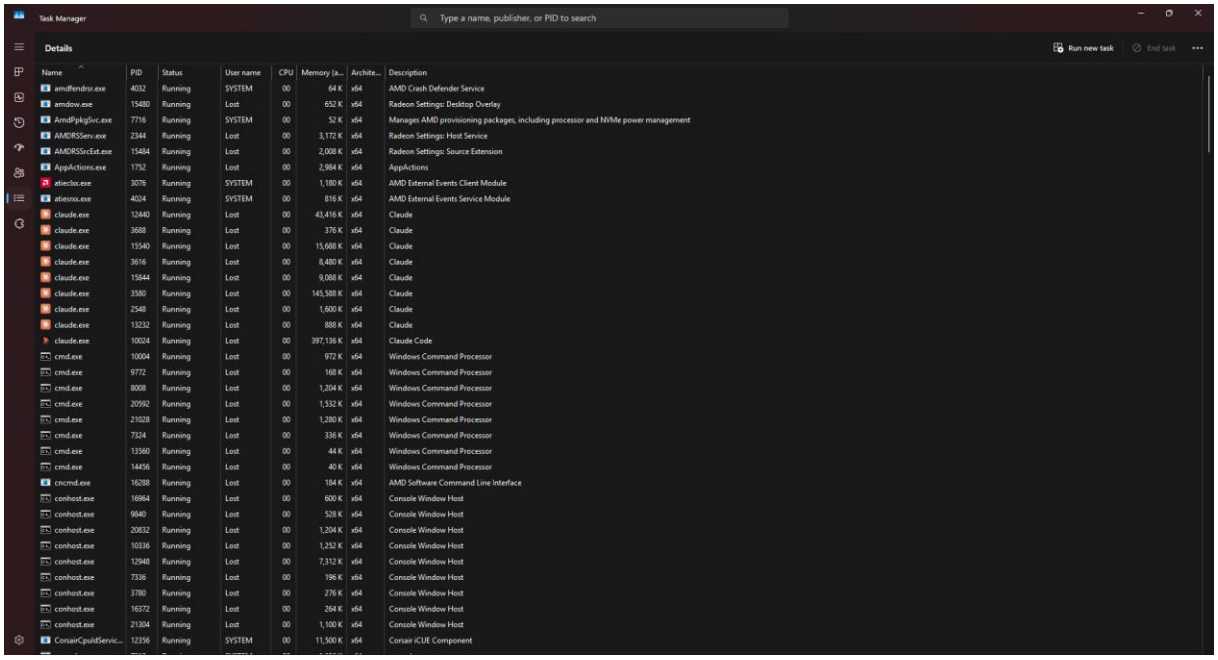


Figura 4. Pestaña Detalles: aquí cada fila es un proceso, con su nombre de imagen, PID, usuario, CPU, memoria y arquitectura.

La guía menciona entre 20 y 30 procesos como referencia para un sistema recién iniciado. Mi equipo está muy por encima porque tiene bastante software de terceros instalado y llevaba horas encendido.

Paso 3. Abrir la línea de comandos

Tecla Windows, escribí cmd, Enter. De aquí en adelante todo lo de Windows sale de comandos escritos.

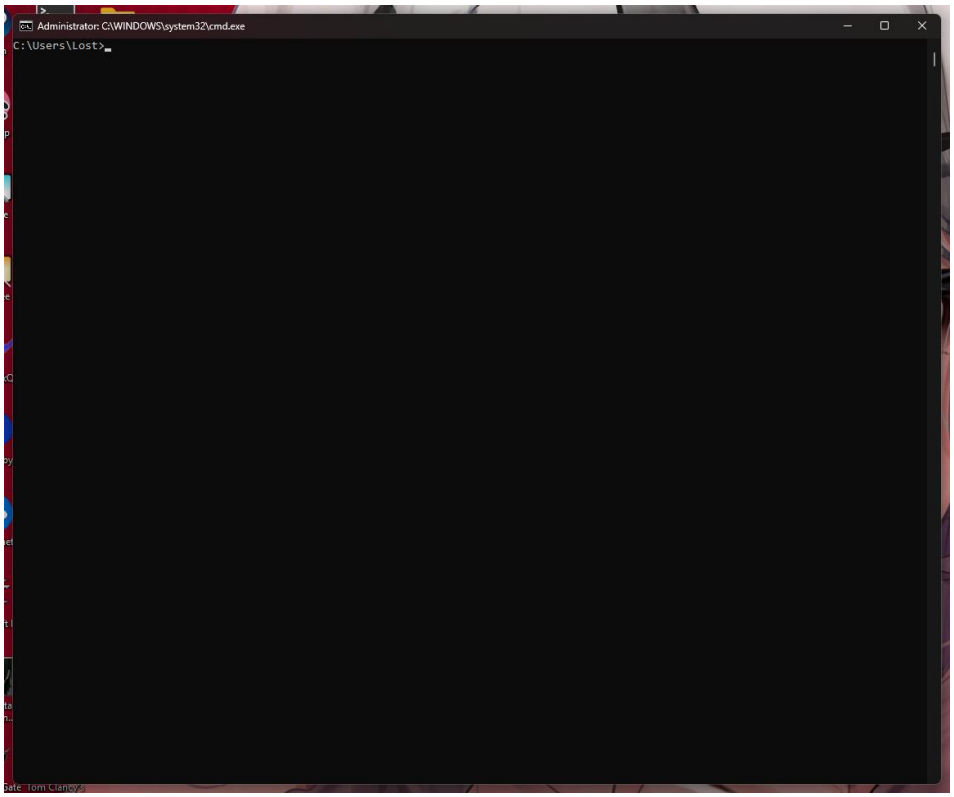


Figura 5. Símbolo del sistema abierto en la carpeta del usuario.

El prompt termina en «>» y muestra la ruta C:\Users\Lost, que era lo que había que comprobar.

Paso 4. Ejecutar systeminfo

systeminfo devuelve un informe largo del equipo. Como no cabía en la ventana, lo pasé por el paginador more para poder leerlo pantalla por pantalla.

```
C:\Users\Lost>systeminfo | more
```

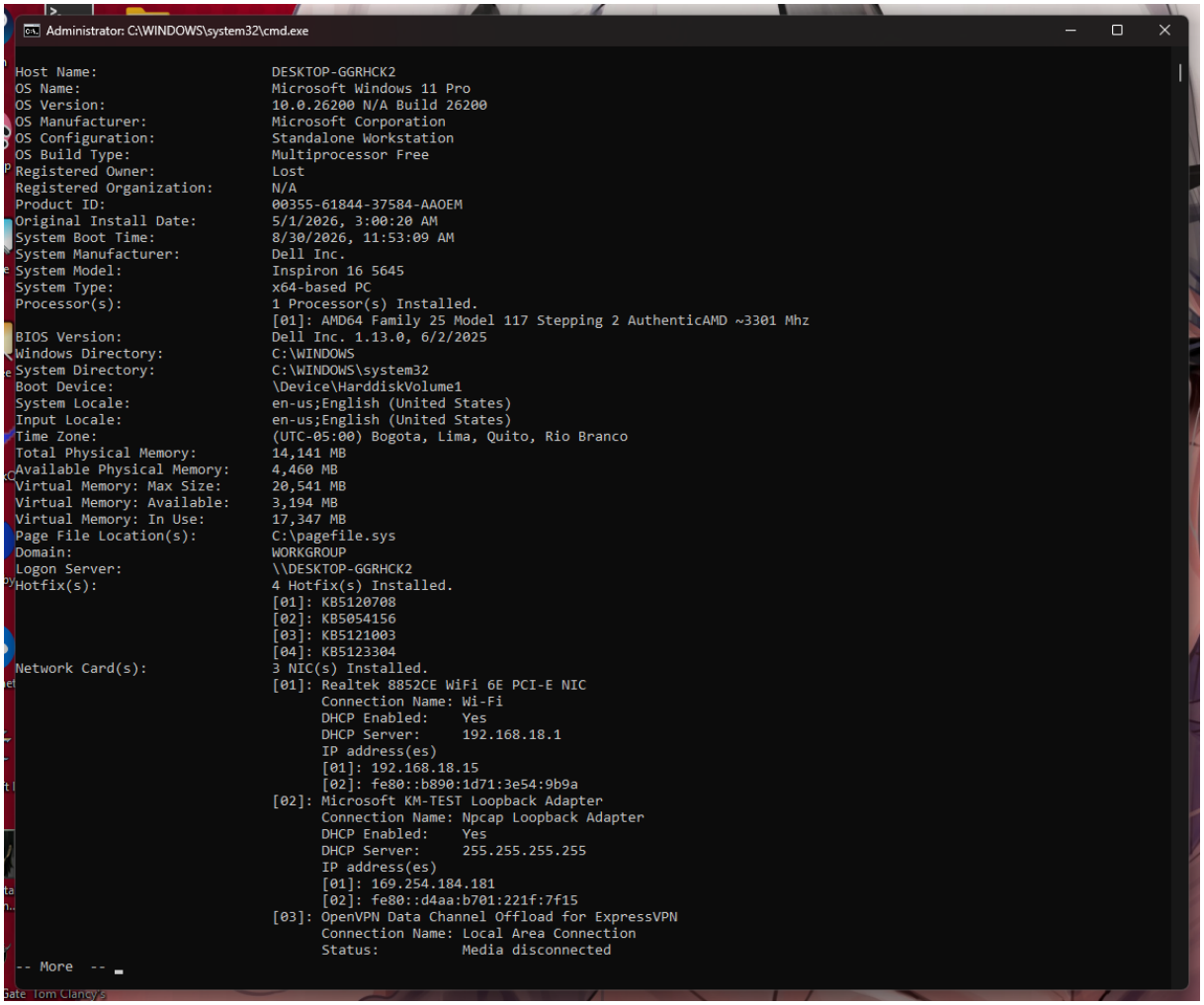


Figura 6. systeminfo, primera pantalla. Aquí están el nombre del sistema, el fabricante, la arquitectura, la memoria total y la hora de arranque.

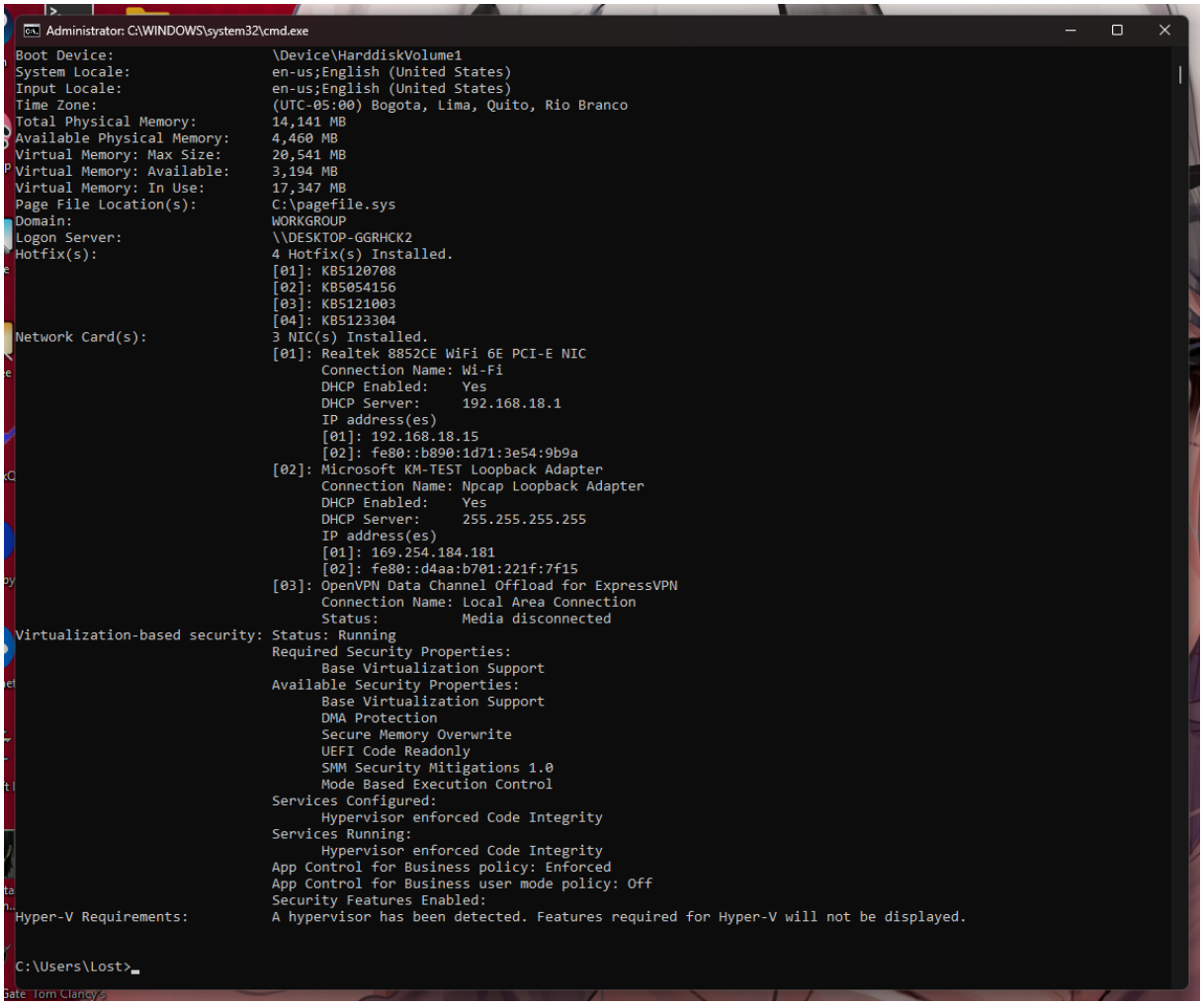


Figura 7. systeminfo, segunda pantalla: memoria virtual, dominio, actualizaciones instaladas, tarjetas de red y seguridad basada en virtualización.

Para tener los datos que pide la guía en una sola vista, repetí el comando filtrando con findstr:

```
C:\Users\Lost>systeminfo | findstr /C:"OS Name" /C:"OS Version" /C:"OS Manufacturer"
        /C:"System Manufacturer" /C:"System Model" /C:"System Type"
        /C:"Total Physical Memory" /C:"System Boot Time"
```

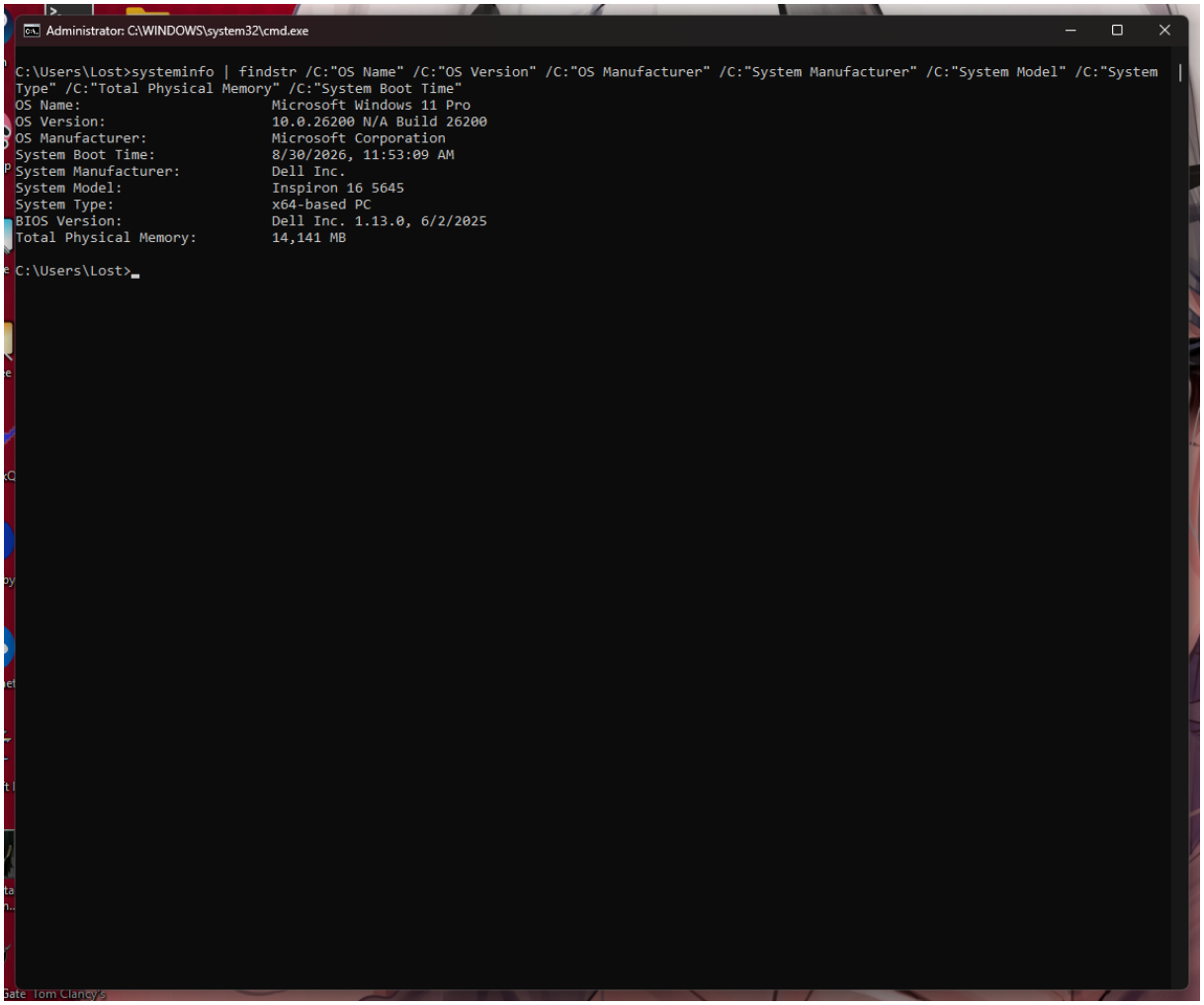


Figura 8. La misma información recortada con findstr a los campos que pide la guía.

Resumen de lo que reporta:

Campo de systeminfo	Valor	Qué es
Host Name	DESKTOP-GGRHCK2	Nombre del equipo en la red
OS Name	Microsoft Windows 11 Pro	Nombre y edición del sistema
OS Version	10.0.26200 N/A Build 26200	Versión interna del núcleo NT
OS Manufacturer	Microsoft Corporation	Fabricante del sistema
OS Configuration	Standalone Workstation	Equipo suelto, sin dominio
System Manufacturer / Model	Dell Inc. / Inspiron 16 5645	Fabricante y modelo del hardware
System Type	x64-based PC	Arquitectura de 64 bits
Processor(s)	AMD64 Family 25 Model 117, AMD Ryzen 7 8840U, ~3301 MHz	Procesador físico
Total Physical Memory	14 141 MB (unos 13,8 GB)	RAM instalada

Campo de systeminfo	Valor	Qué es
Available Physical Memory	4 460 MB	RAM libre al momento de medir
System Boot Time	30/08/2026, 11:53:09 a. m.	Último arranque, tema 1.6
BIOS Version	Dell Inc. 1.13.0, 02/06/2025	Firmware del equipo
Time Zone	(UTC-05:00) Bogotá, Lima, Quito	Zona horaria

Los tres campos que la verificación pedía localizar (OS Name, System Type y System Boot Time) están todos en la salida.

Paso 5. Ejecutar tasklist

tasklist hace por comandos lo que el Administrador de tareas hace por ventanas: saca todos los procesos con su PID, la sesión donde corren y la memoria que ocupan. También lo paginé.

```
C:\Users\Lost>tasklist | more
```

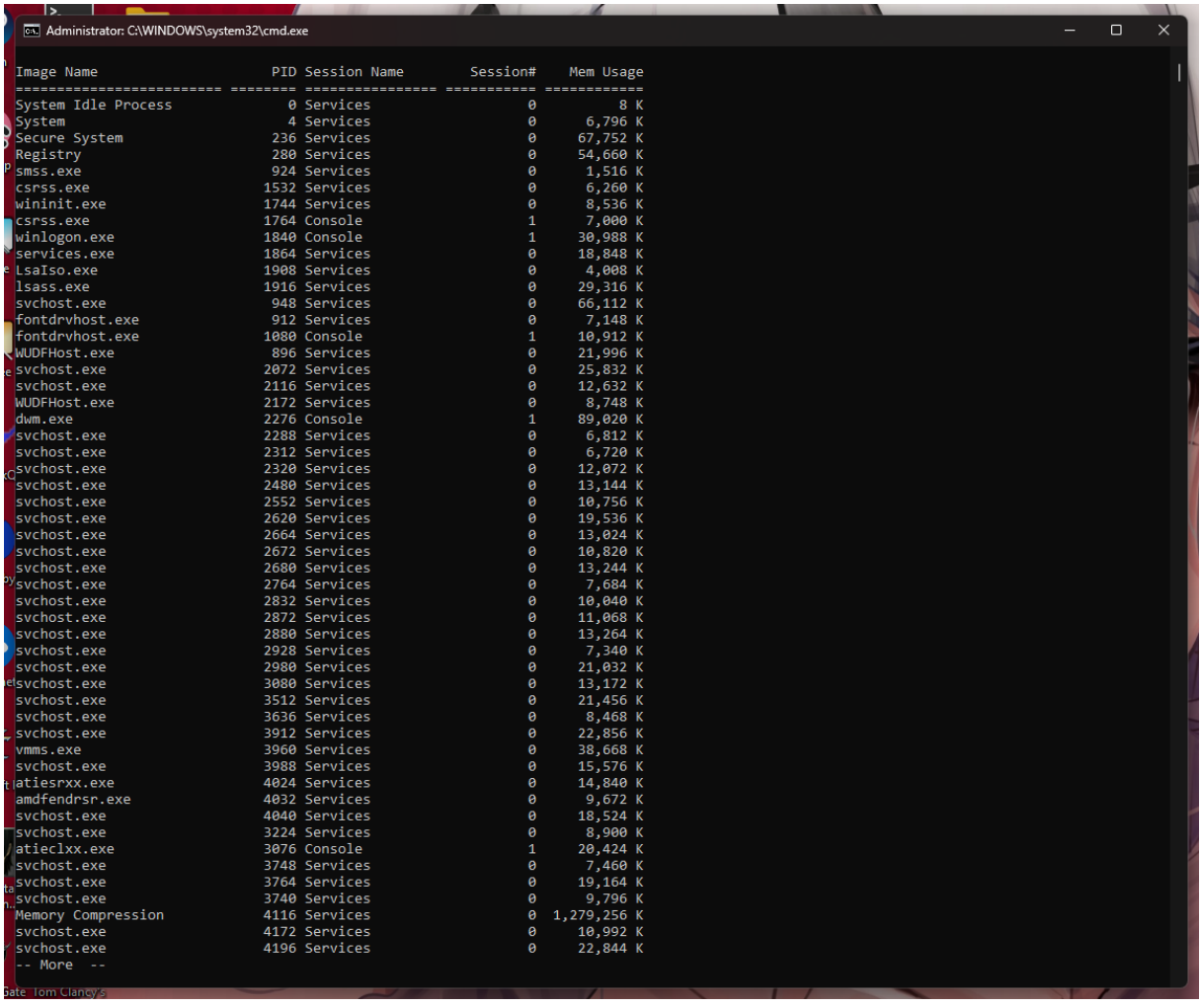


Figura 9. tasklist, primera pantalla. Se ven los procesos con los que arranca el sistema: System Idle Process con PID 0, System con PID 4, smss.exe, csrss.exe, wininit.exe, winlogon.exe, services.exe, lsass.exe y una buena cantidad de svchost.exe.

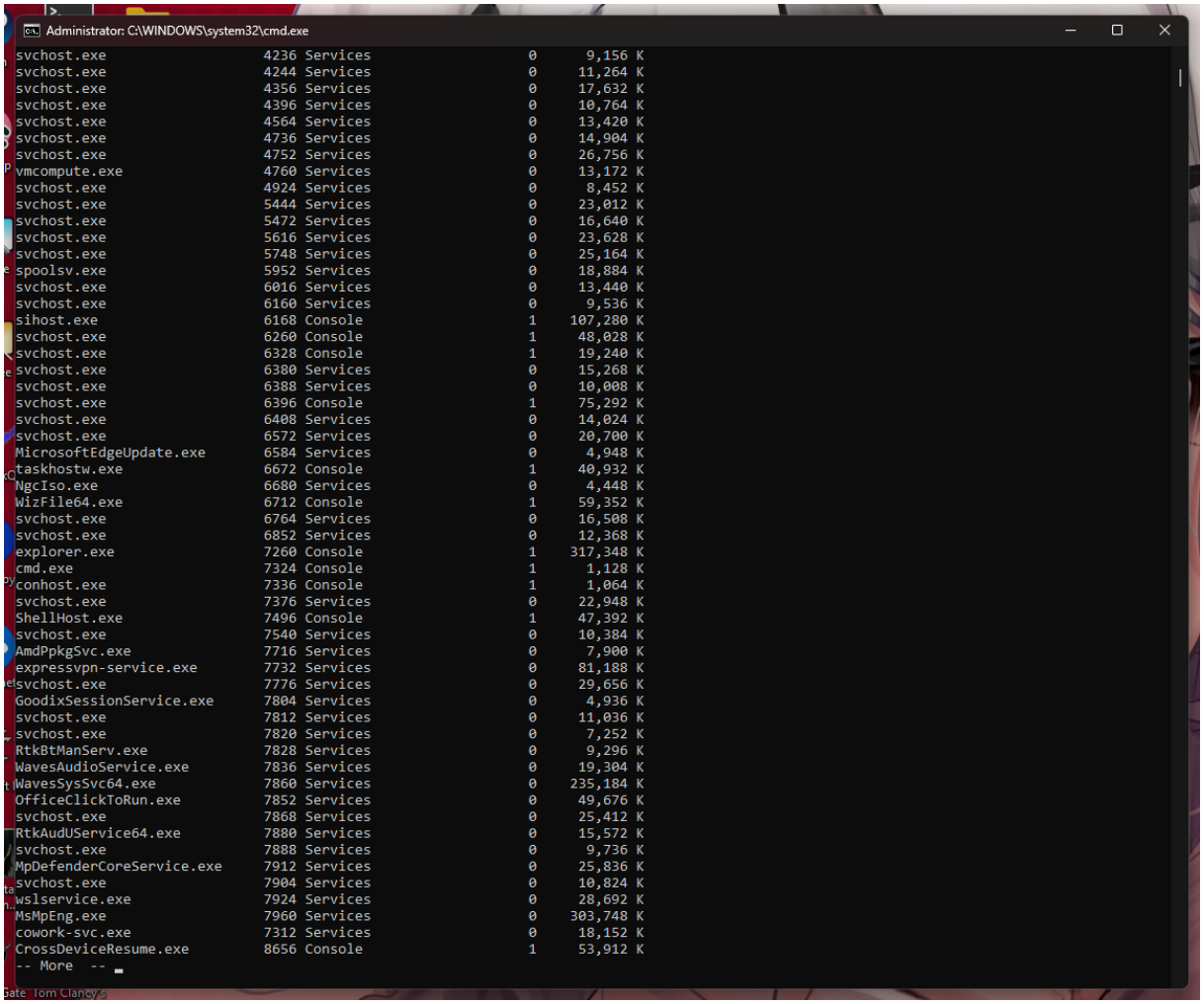


Figura 10. tasklist, pantalla intermedia: servicios y aplicaciones de usuario, cada uno con su PID y su consumo de memoria.

La cantidad de filas se parece a lo que había visto en el paso 2. El número exacto sale en el paso siguiente.

Paso 6. Contar los procesos con una tubería

La barra vertical, el pipe, agarra lo que tasklist iba a imprimir en pantalla y se lo pasa a find como entrada. Con /c le pido que cuente en vez de mostrar, y con /v "" que se quede con las líneas que no están vacías. El resultado es un solo número.

```
C:\Users\Lost>tasklist | find /c /v ""
276
```

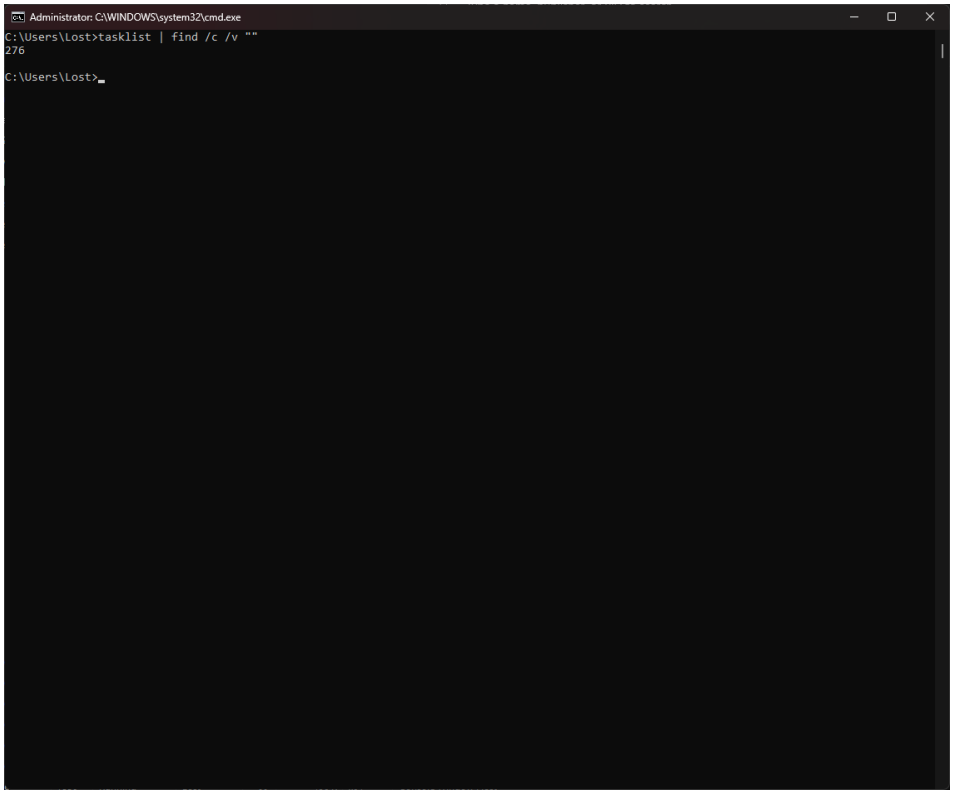


Figura 11. El conteo de procesos de Windows: 276 líneas.

Concepto	**Valor**
Líneas que devuelve tasklist | find /c /v ""	276
Encabezados de tasklist (título, separador y línea en blanco)	3
Procesos reales	273

276 menos 3 son 273 procesos, contra las 208 entradas que mostraba el Administrador de tareas. Los números no cuadran y esa diferencia es el tema de la primera pregunta de reflexión.

Parte B. Linux por línea de comandos en una VM importada

Acá el trabajo es al revés que en la parte anterior. Hay que importar un disco virtual que ya trae Linux instalado, darle recursos, arrancarlo y revisar el sistema entero sin abrir una sola ventana dentro del invitado.

Paso 7. Crear la VM con el disco .vdi existente

Botón «Nueva» en VirtualBox. En la primera sección del asistente puse el nombre y el sistema operativo invitado: nombre Lab1-Linux, tipo Linux, distribución Ubuntu, versión Ubuntu (64-bit).

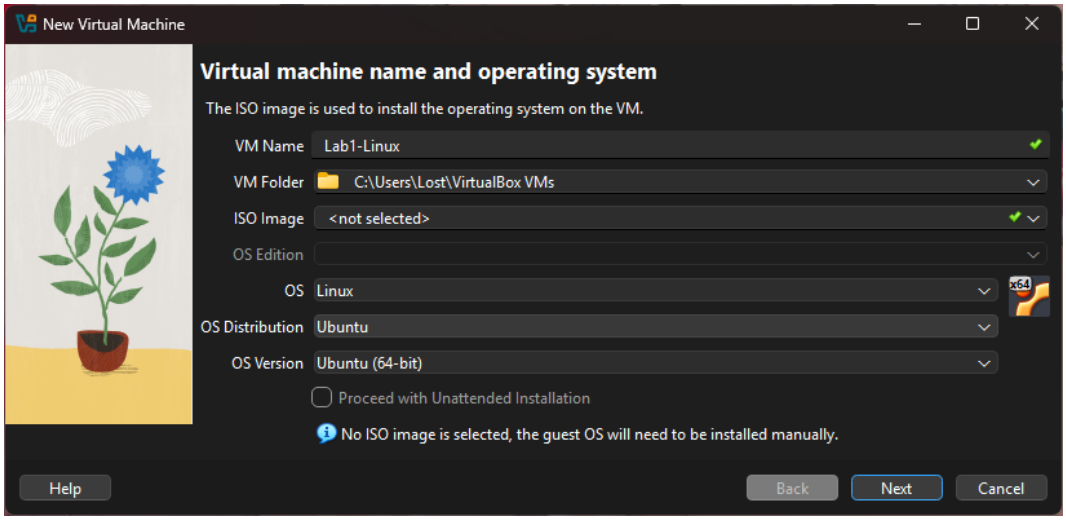


Figura 12. Asistente de máquina virtual nueva, con el nombre y el sistema operativo ya elegidos.

En la sección del disco duro hay que tener cuidado de no dejar la opción por defecto. Marqué «Usar un archivo de disco duro virtual existente» y busqué el .vdi, en lugar de crear uno nuevo:

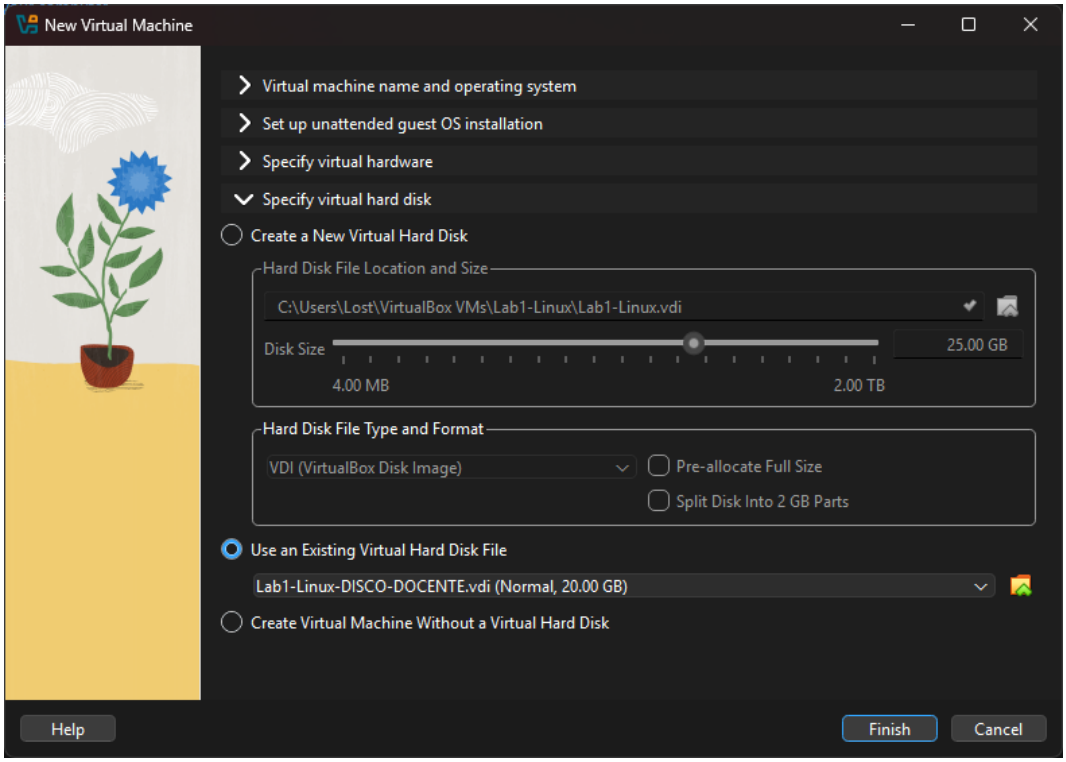


Figura 13. Disco seleccionado: Lab1-Linux-DISCO-DOCENTE.vdi (Normal, 20,00 GB), con el botón de radio en «Use an Existing Virtual Hard Disk File».

El asistente muestra el archivo con su tamaño antes de terminar, que era la comprobación del paso. Al pulsar Finalizar, la máquina queda registrada con el disco colgado del controlador SATA.

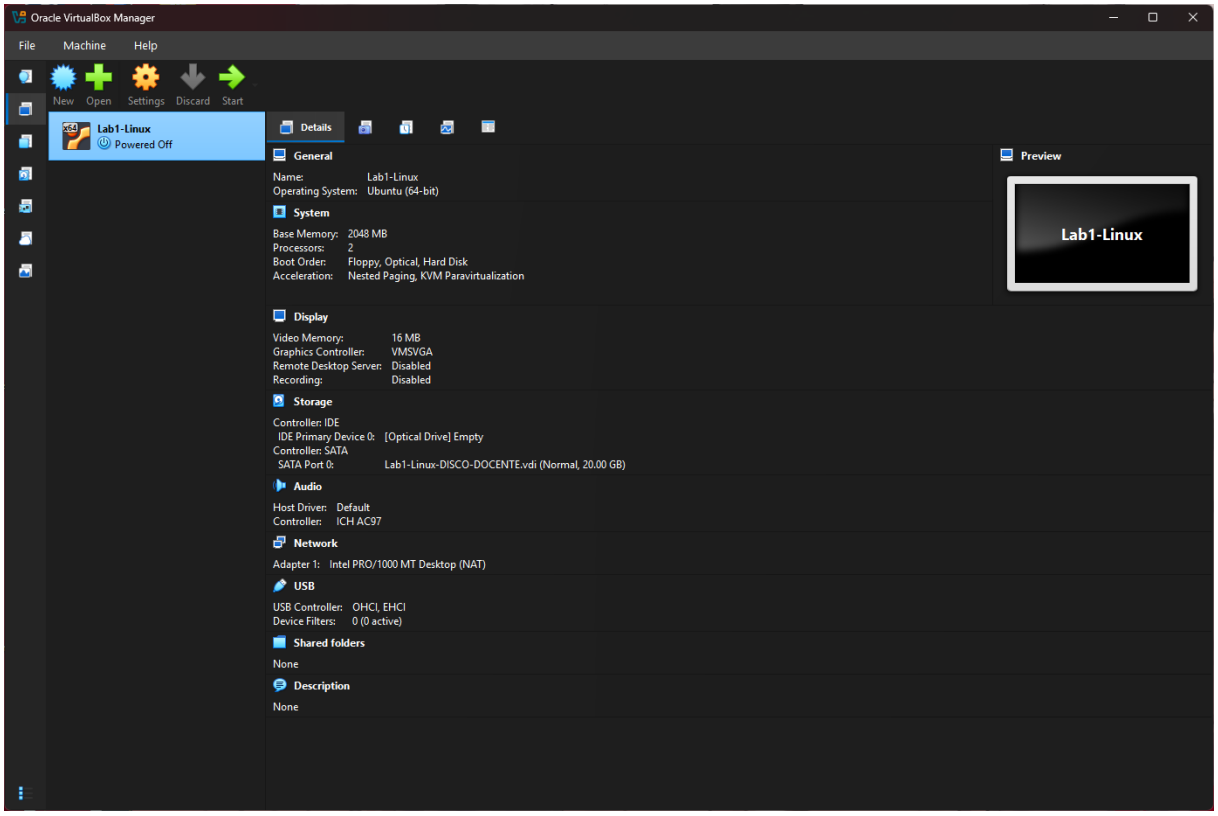


Figura 14. VirtualBox Manager con la VM ya creada. En Almacenamiento aparece SATA Puerto 0 con el .vdi de 20,00 GB.

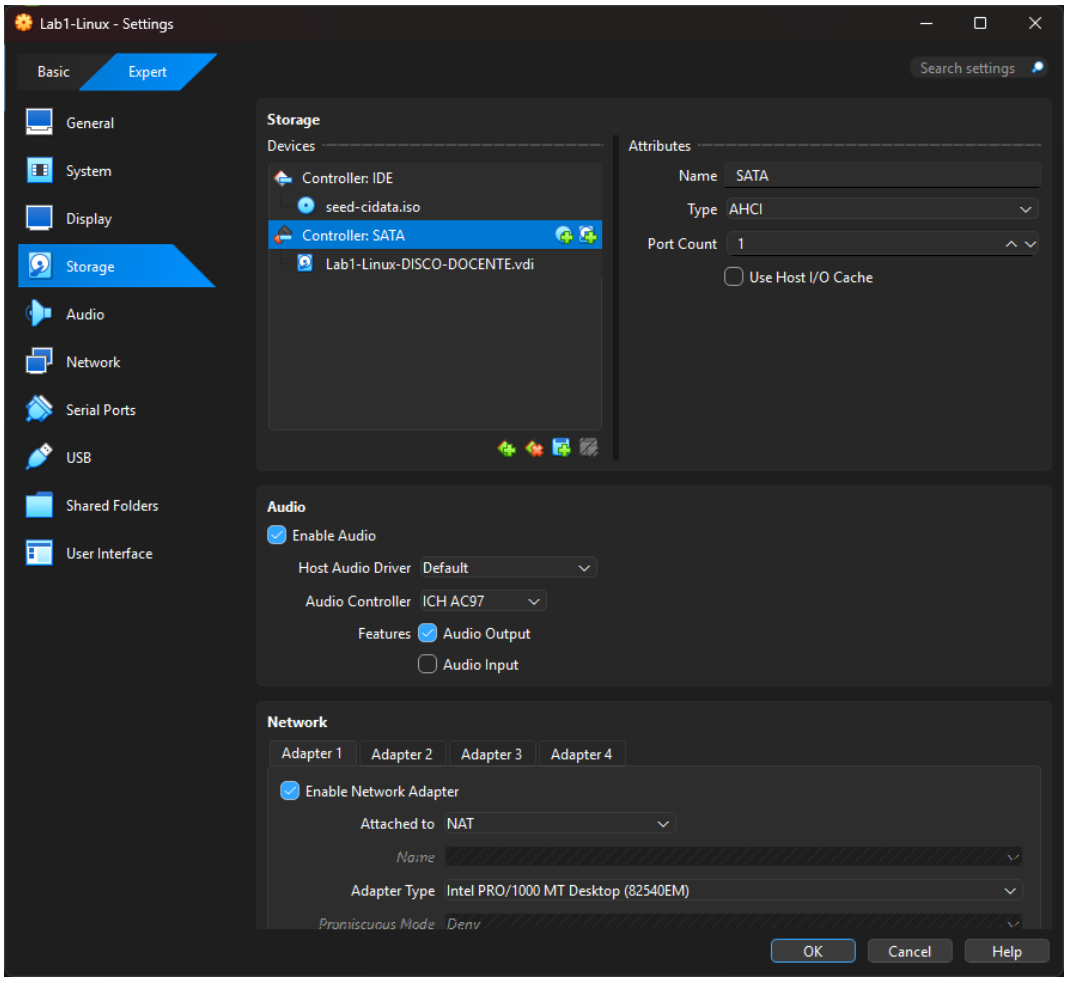


Figura 15. Configuración, sección Almacenamiento: el .vdi en el controlador SATA y el medio con las credenciales de cloud-init en el IDE.

Paso 8. Ajustar los recursos de la máquina

Con la VM seleccionada entré a Configuración, sección Sistema. La pestaña Placa base tiene la memoria base y la pestaña Procesador la cantidad de CPU. Los 2048 MB ya venían puestos desde el asistente; las CPU había que subirlas de 1 a 2.

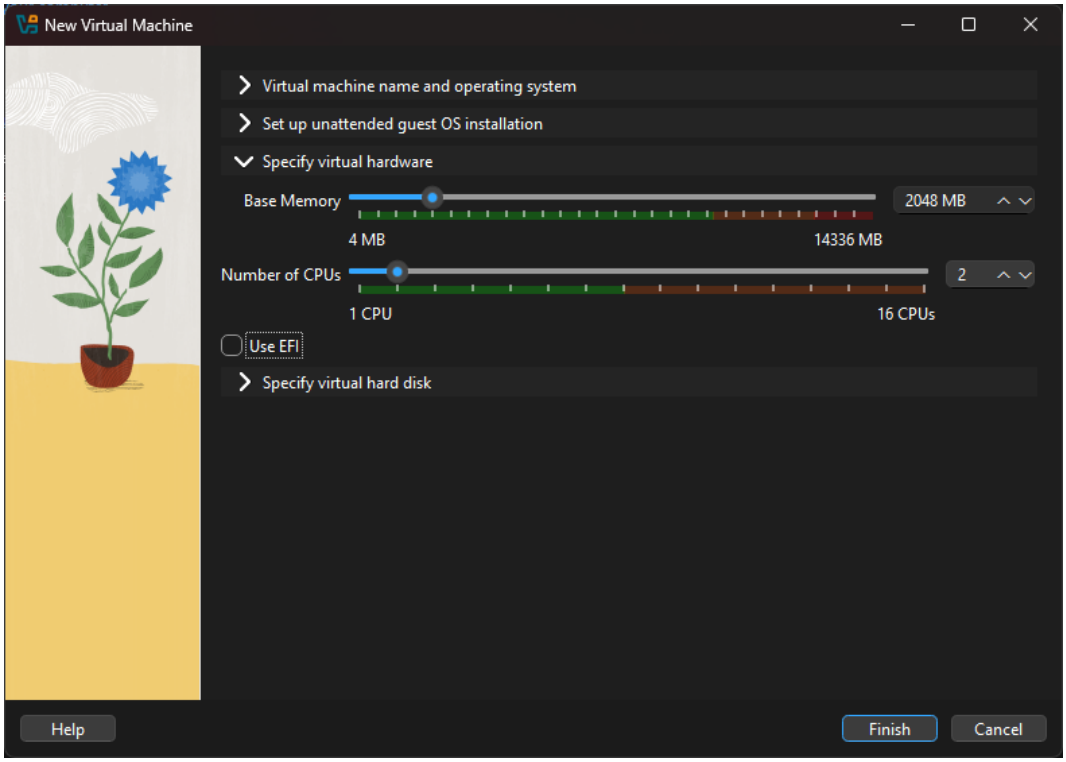


Figura 16. Recursos definidos durante la creación: 2048 MB de memoria base y 2 CPU.

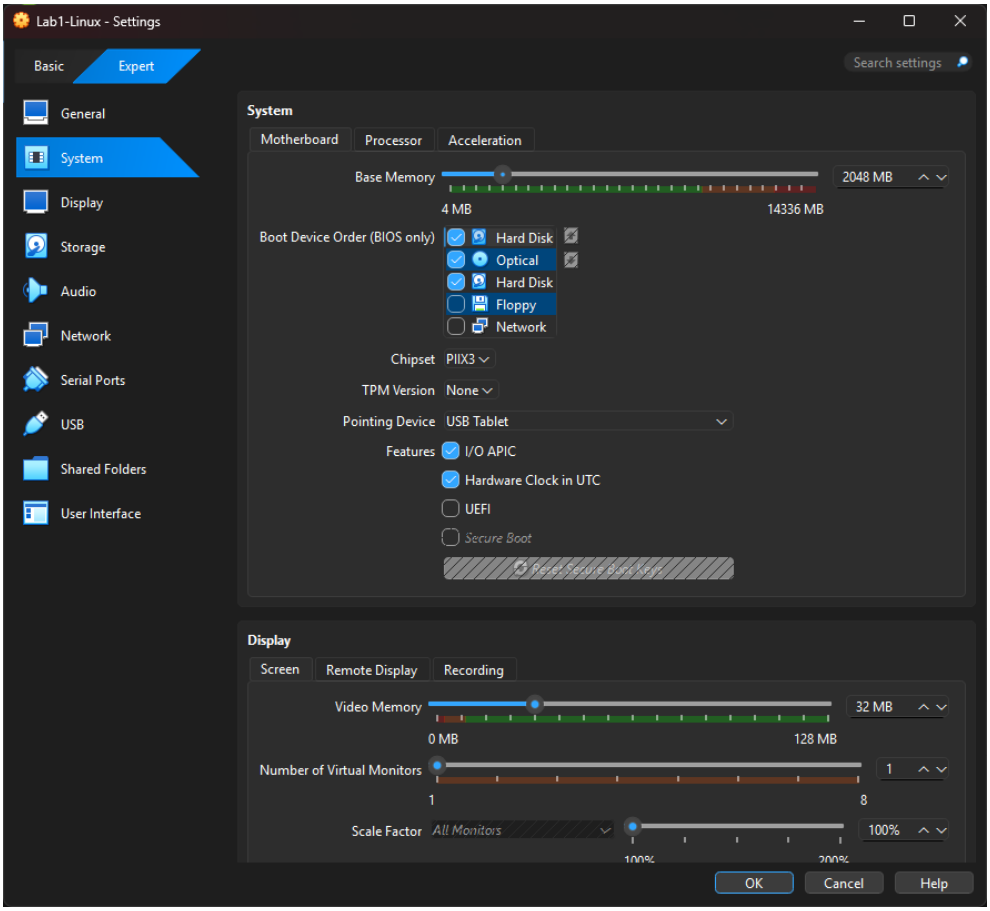


Figura 17. Configuración, Sistema, pestaña Placa base: memoria base en 2048 MB.

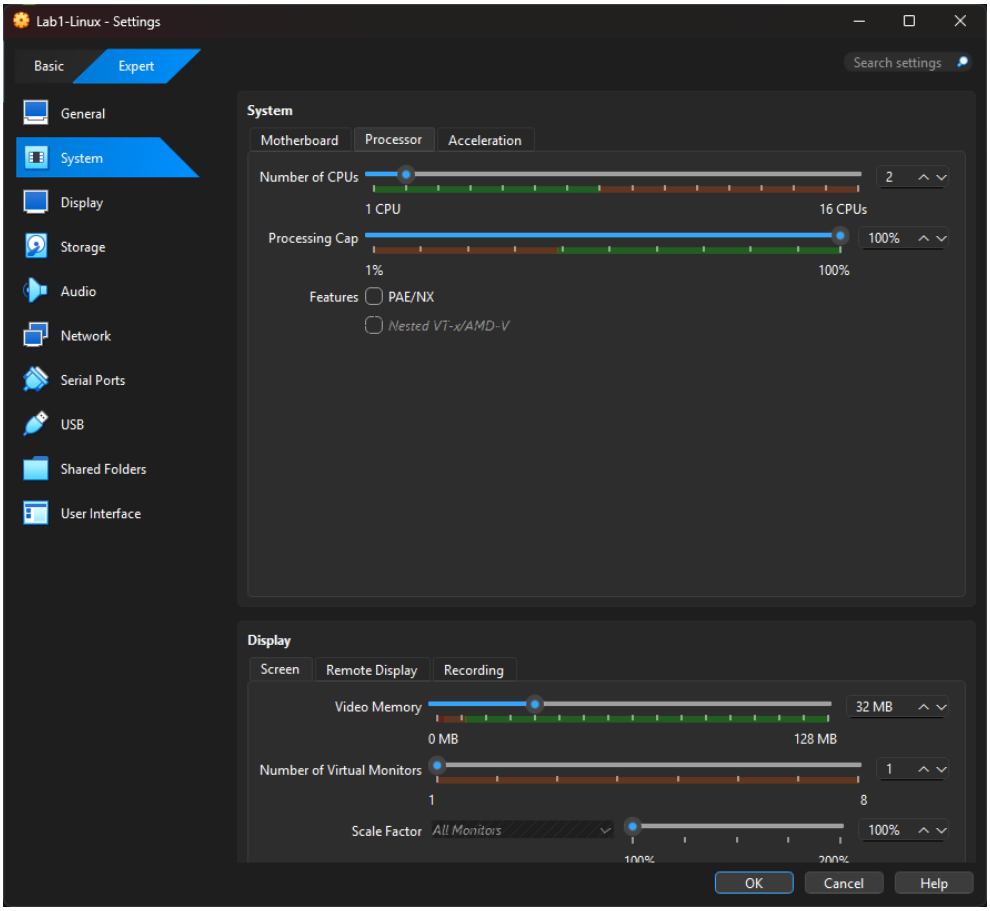


Figura 18. Configuración, Sistema, pestaña Procesador: 2 CPU.

Recurso	Valor	Lo que pedía la guía
Memoria base	2048 MB	Al menos 2048 MB
Procesadores	2 CPU	2 CPU si el equipo lo permite
Disco duro virtual	Lab1-Linux-DISCO-DOCENTE.vdi, 20,00 GB en SATA	Un disco ya existente
Memoria de video	32 MB	No se especifica
Red	Adaptador 1, Intel PRO/1000 MT Desktop en NAT	No se especifica

Al cerrar el diálogo los valores quedaron guardados y no salió ningún mensaje en rojo.

Paso 9. Arrancar la VM e iniciar sesión

Seleccioné Lab1-Linux y pulsé Iniciar. Arrancó desde el disco virtual, pasó los mensajes del kernel y se quedó en el login de texto de tty1. No hay escritorio ni nada parecido: es una instalación de servidor.

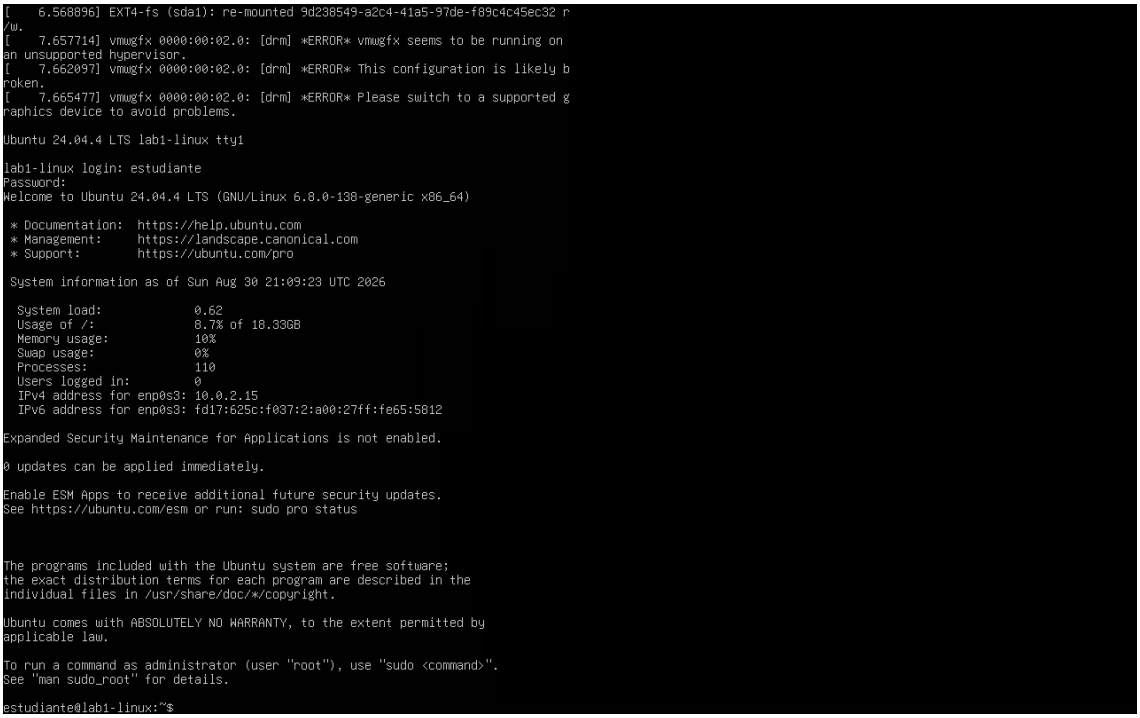


Figura 19. Sesión iniciada con el usuario estudiante. El mensaje del día ya adelanta bastante información: Ubuntu 24.04.4 LTS, kernel 6.8.0-138-generic, 8,7 % de uso de un disco de 18,33 GB, 110 procesos y la dirección IPv4 10.0.2.15.

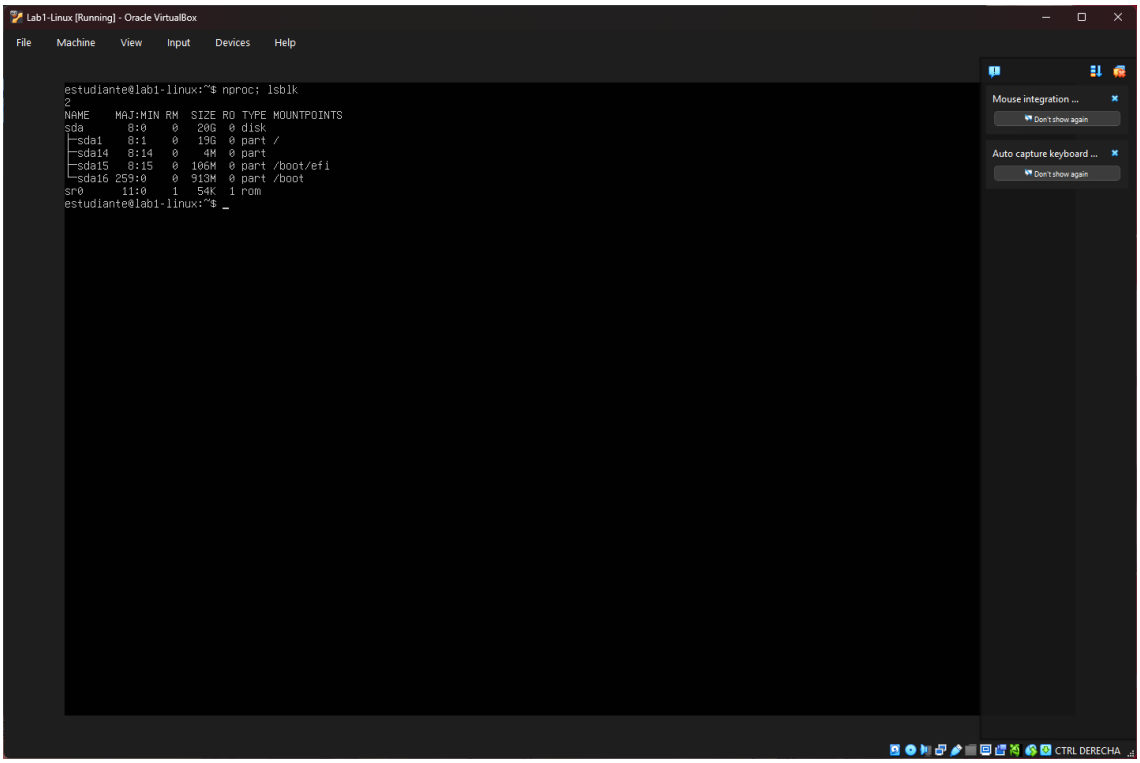


Figura 20. La misma sesión vista desde afuera, dentro de la ventana de VirtualBox en Windows.

El prompt quedó como estudiante@lab1-linux:~$, con el formato usuario@equipo que pedía la verificación.

Paso 10. Identificar el sistema con uname -a

```
estudiante@lab1-linux:~$ uname -a
Linux lab1-linux 6.8.0-138-generic #138-Ubuntu SMP PREEMPT_DYNAMIC
Fri Jul 31 22:41:49 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
```

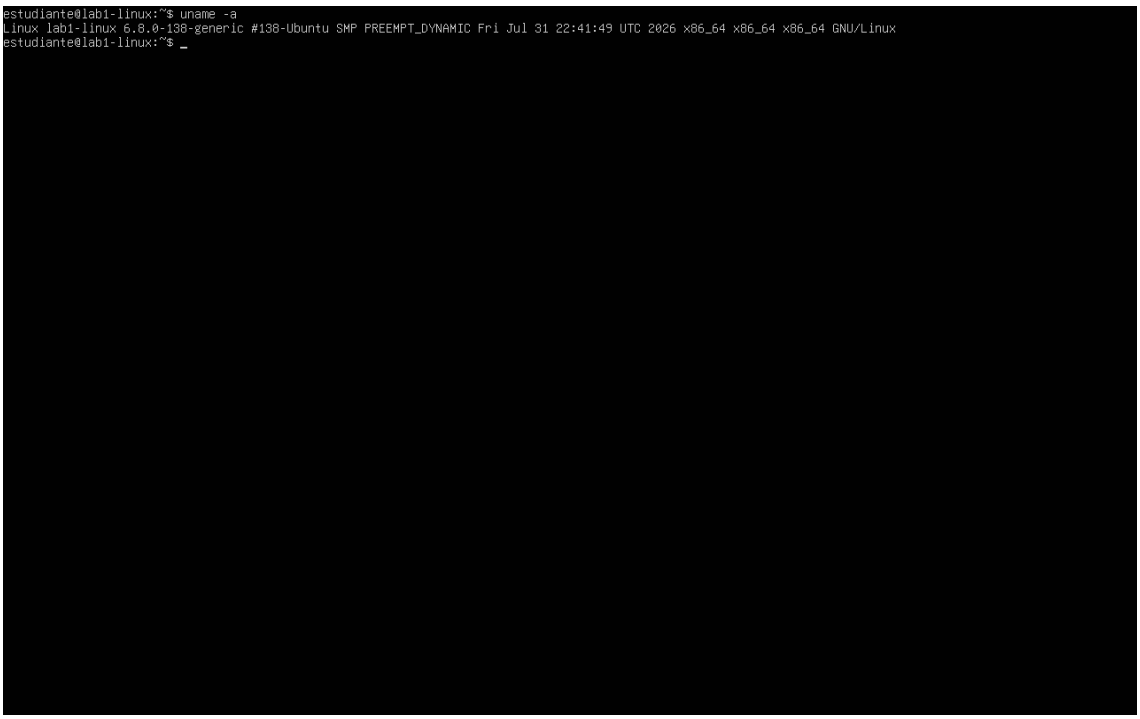


Figura 21. Salida de uname -a dentro de la máquina virtual.

Toda esa línea son siete datos pegados:

Trozo de la salida	Qué significa
Linux	Nombre del kernel
lab1-linux	Nombre del equipo
6.8.0-138-generic	Versión del kernel
#138-Ubuntu SMP PREEMPT_DYNAMIC	Número de compilación; SMP indica soporte multiprocesador
Fri Jul 31 22:41:49 UTC 2026	Cuándo se compiló ese kernel
x86_64	Arquitectura de 64 bits
GNU/Linux	Sistema operativo

Empieza con la palabra Linux, que era lo que había que verificar.

Paso 11. Ver la distribución con cat /etc/os-release

Esto me pareció lo más raro al principio. Para saber qué distribución tengo no ejecuto un programa que me lo diga: leo un archivo de texto con cat, el mismo comando con el que leería cualquier otra cosa. En Linux la identidad del sistema también es un archivo.

```
estudiante@lab1-linux:~$ cat /etc/os-release
PRETTY_NAME="Ubuntu 24.04.4 LTS"
NAME="Ubuntu"
VERSION_ID="24.04"
VERSION="24.04.4 LTS (Noble Numbat)"
VERSION_CODENAME=noble
ID=ubuntu
ID_LIKE=debian
HOME_URL="https://www.ubuntu.com/"
SUPPORT_URL="https://help.ubuntu.com/"
BUG_REPORT_URL="https://bugs.launchpad.net/ubuntu/"
UBUNTU_CODENAME=noble
```



Figura 22. Contenido de /etc/os-release.

Ahí están las dos líneas que pedía la verificación, NAME="Ubuntu" y VERSION="24.04.4 LTS (Noble Numbat)". De paso, ID_LIKE=debian dice de dónde viene Ubuntu.

Comprobación extra con hostnamectl

hostnamectl junta en una sola salida el sistema, el kernel y el hardware. Lo corrí porque me sirve para la segunda pregunta de reflexión: dice sin rodeos que esto no es una máquina física.

```
estudiante@lab1-linux:~$ hostnamectl
 Static hostname: lab1-linux
       Icon name: computer-vm
         Chassis: vm
  Virtualization: oracle
Operating System: Ubuntu 24.04.4 LTS
          Kernel: Linux 6.8.0-138-generic
    Architecture: x86-64
 Hardware Vendor: innotek GmbH
  Hardware Model: VirtualBox
```

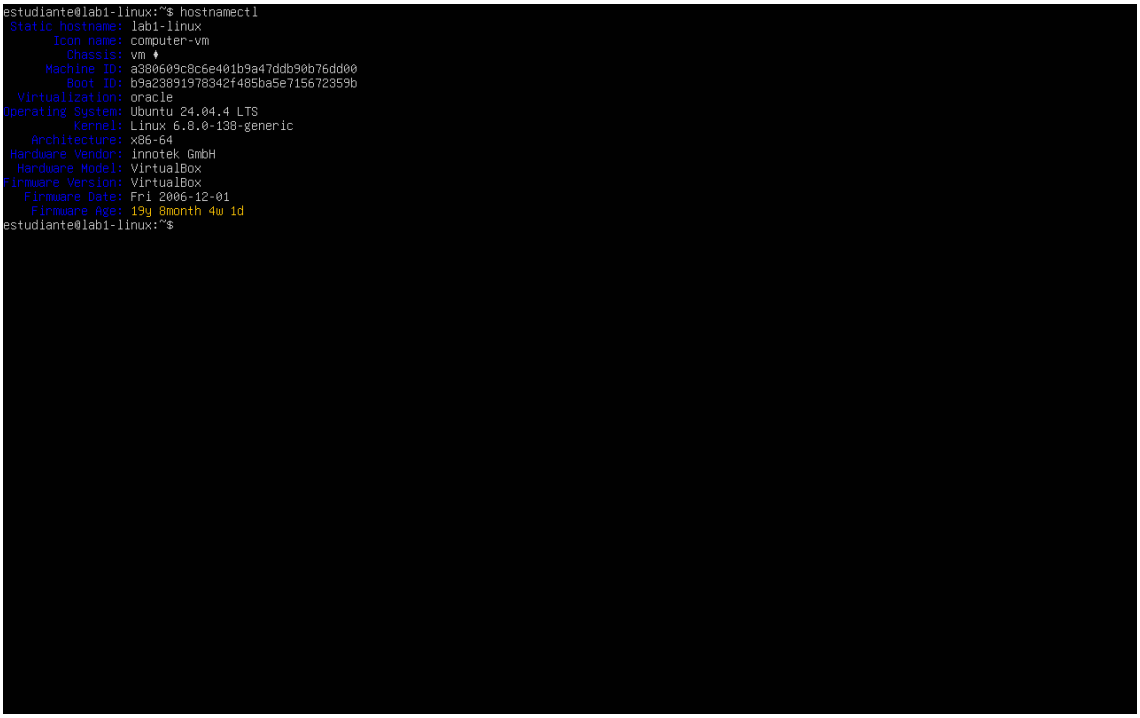


Figura 23. hostnamectl. Las líneas Virtualization: oracle y Hardware Model: VirtualBox son las que delatan al hipervisor.

Paso 12. Confirmar usuario y grupos con whoami e id

```
estudiante@lab1-linux:~$ whoami
estudiante
estudiante@lab1-linux:~$ id
uid=1000(estudiante) gid=1000(estudiante) groups=1000(estudiante),4(adm),
24(cdrom),27(sudo),30(dip),46(plugdev),100(users)
```

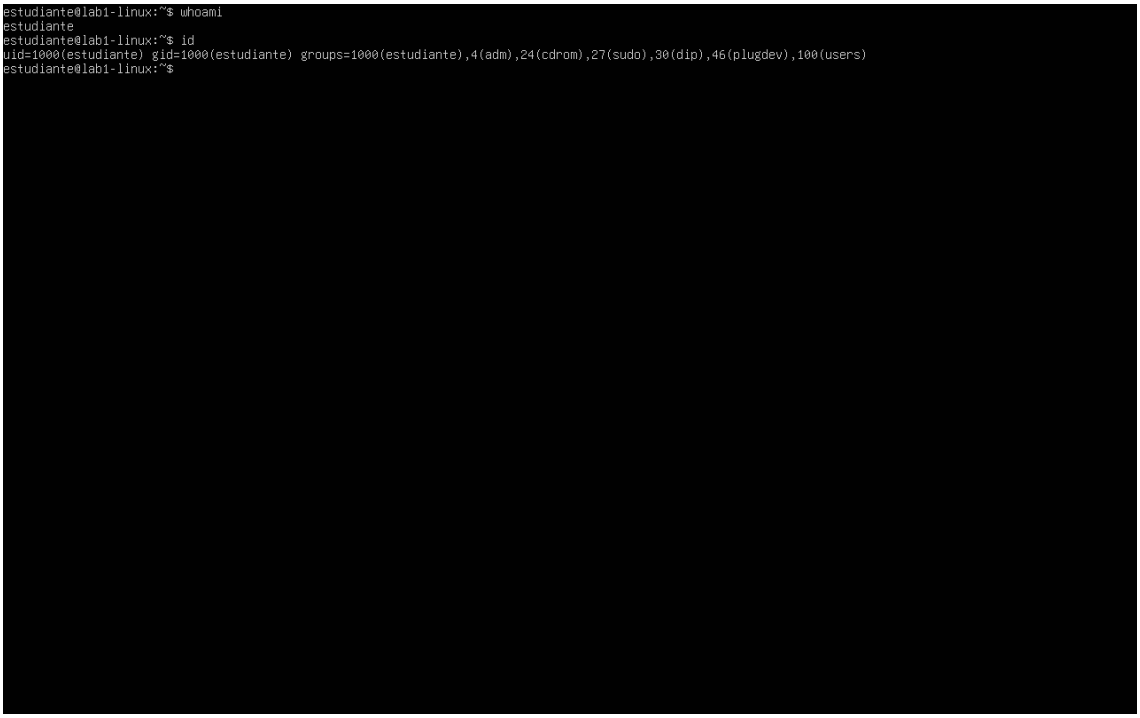


Figura 24. Salidas de whoami e id.

Dato	Valor	Qué significa
Usuario	estudiante	El mismo con el que entré en el paso 9
UID	1000	Primer usuario normal; los menores a 1000 son del sistema
GID principal	1000 (estudiante)	Su grupo propio
Grupos secundarios	adm (4), cdrom (24), sudo (27), dip (30), plugdev (46), users (100)	Estar en sudo es lo que habilita las tareas de administración

whoami devuelve el mismo nombre con el que inicié sesión, así que la verificación cierra.

Paso 13. Contar los procesos con ps

ps aux saca todos los procesos con bastante detalle: quién es el dueño, cuánta CPU y cuánta memoria usa, en qué estado está y con qué línea de comando arrancó. wc -l cuenta las líneas que salieron.

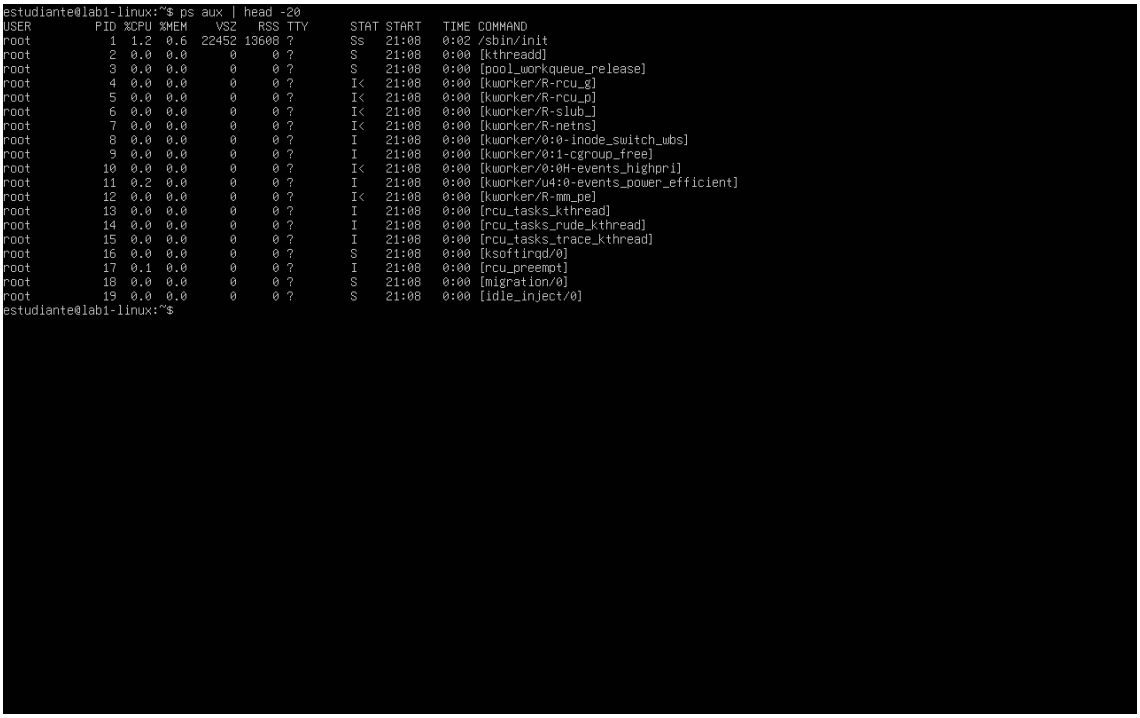


Figura 25. Las primeras veinte líneas de ps aux. Arriba del todo está /sbin/init con PID 1, el primer proceso que crea el kernel, y debajo los hilos internos del propio kernel, que se muestran entre corchetes.

```
estudiante@lab1-linux:~$ ps aux | wc -l
112
```



Figura 26. El conteo de procesos en Linux: 112 líneas.

Concepto	Valor
Líneas que devuelve ps aux | wc -l	112
Encabezado de ps aux	1
Procesos reales en la VM	111

La comparación que pide el paso

Sistema	Comando	Resultado bruto	Procesos
Windows 11 Pro (anfitrión)	tasklist | find /c /v ""	276	273
Ubuntu 24.04.4 LTS (VM)	ps aux | wc -l	112	111

Gana Windows por bastante: 273 contra 111. No creo que eso diga gran cosa sobre eficiencia. El anfitrión es una máquina de uso diario, con navegador, mensajería, antivirus, utilidades del fabricante y el propio VirtualBox corriendo encima. La VM es un servidor recién arrancado, sin escritorio y sin nada instalado por mí. Lo que sí vale la pena mirar es que hasta ese sistema pelado pasa de los cien procesos.

Paso 14. Revisar memoria y disco con free -h y df -h

```
estudiante@lab1-linux:~$ free -h
               total        used        free      shared  buff/cache   available
Mem:           1.9Gi       327Mi       1.4Gi       1.0Mi       326Mi       1.6Gi
Swap:             0B          0B          0B
```

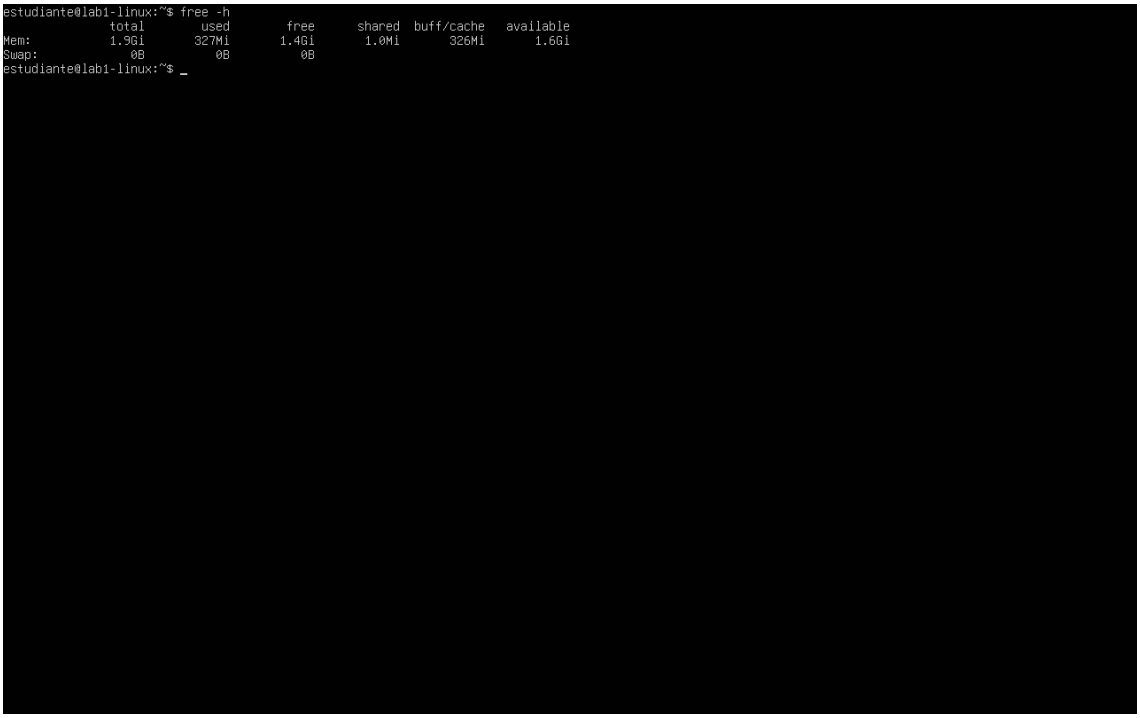


Figura 27. free -h: memoria de la máquina virtual.

Los 1,9 GiB son los 2048 MB que asigné en el paso 8, menos lo que el kernel se reserva para sí mismo. La fila Swap en 0B llama la atención: esta imagen viene sin partición de intercambio.

```
estudiante@lab1-linux:~$ df -h
Filesystem      Size  Used Avail Use% Mounted on
tmpfs           197M  1.1M  196M   1% /run
/dev/sda1        19G  1.7G   17G   9% /
tmpfs           984M     0  984M   0% /dev/shm
tmpfs           5.0M     0  5.0M   0% /run/lock
/dev/sda16      881M   64M  756M   8% /boot
/dev/sda15      105M  6.2M   99M   6% /boot/efi
tmpfs           197M   12K  197M   1% /run/user/1000
```

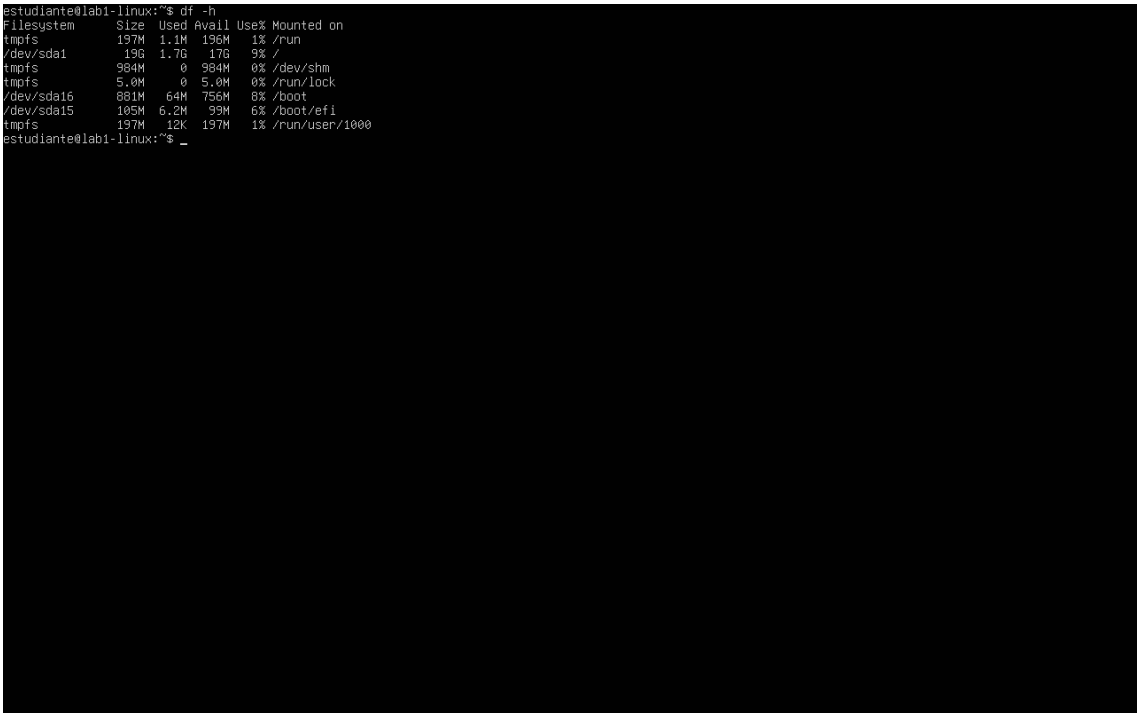


Figura 28. df -h: espacio por sistema de archivos montado.

Sistema de archivos	Tamaño	Usado	Libre	Uso	Montado en
/dev/sda1	19G	1.7G	17G	9%	/ (raíz)
/dev/sda16	881M	64M	756M	8%	/boot
/dev/sda15	105M	6.2M	99M	6%	/boot/efi
tmpfs	984M	0	984M	0%	/dev/shm, vive en memoria
tmpfs	197M	1.1M	196M	1%	/run

free muestra las filas Mem y Swap, y df tiene su línea con la raíz montada en /dev/sda1. Las dos comprobaciones del paso quedan cubiertas.

Comprobación extra con nproc y lsblk

```
estudiante@lab1-linux:~$ nproc; lsblk
2
NAME    MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda       8:0     0   20G  0 disk
|-sda1    8:1     0   19G  0 part /
|-sda14   8:14    0    4M  0 part
|-sda15   8:15    0  106M  0 part /boot/efi
`-sda16 259:0     0  913M  0 part /boot
sr0      11:0     1   54K  1 rom
```

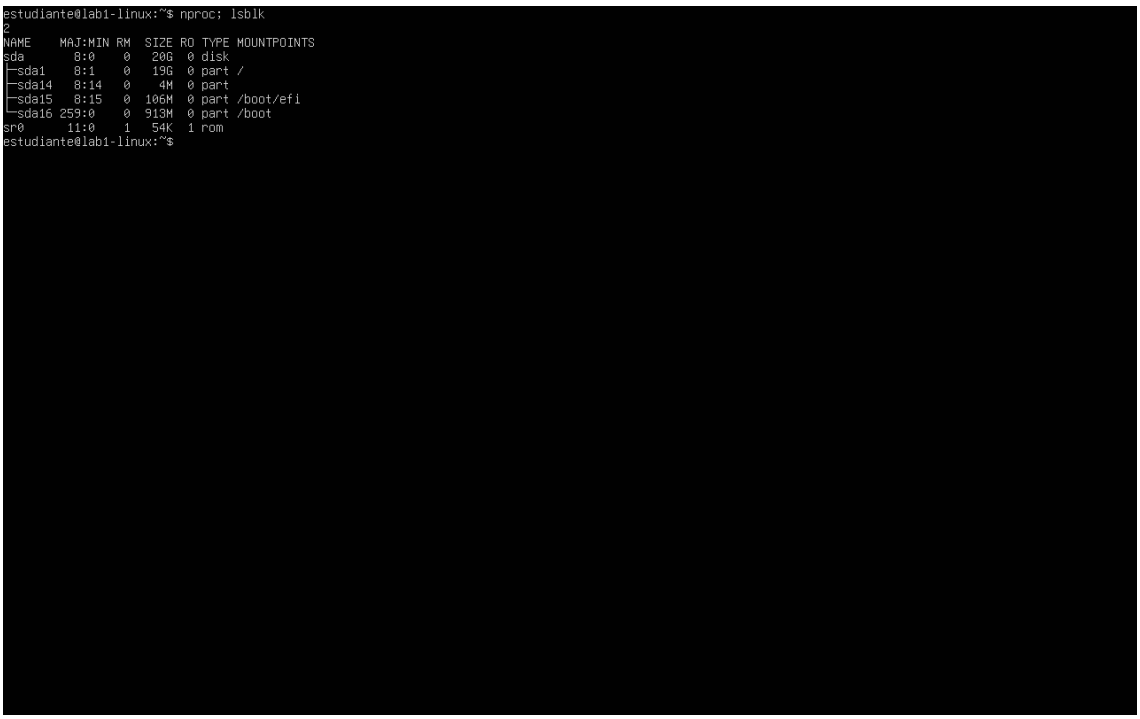


Figura 29. nproc devuelve las 2 CPU que asigné en el paso 8 y lsblk muestra el disco sda de 20 GB, que no es otra cosa que el archivo .vdi del paso 7.

Tabla comparativa entre Windows y Linux

Todos los valores de esta tabla salen de los comandos que aparecen en las dos partes anteriores. No hay nada tomado de internet.

Característica	Windows (Parte A)	Linux (Parte B)
Nombre y versión	Microsoft Windows 11 Pro, versión 25H2, compilación 26200.9168. Núcleo NT 10.0.26200	Ubuntu 24.04.4 LTS «Noble Numbat», kernel Linux 6.8.0-138-generic
Quién lo desarrolla	Microsoft Corporation. Software propietario, con licencia comercial	Canonical y la comunidad. El kernel es software libre bajo licencia GPL
Arquitectura	x64 de 64 bits, según el campo System Type de systeminfo	x86_64 de 64 bits, según uname -a y hostnamectl
Tipo según 1.3	Multiusuario y multitarea expropiativa, de propósito general. En la práctica se usa como sistema de un solo usuario interactivo y se administra por ventanas	Multiusuario y multitarea expropiativa, de propósito general. Esta instalación es de servidor y se administra escribiendo comandos
Generación aproximada según 1.2	Cuarta y quinta generación. Viene de la familia Windows NT, que arranca en 1993. Windows 11 salió en 2021	Cuarta y quinta generación. Desciende de UNIX (1969) por la vía del kernel de Linus Torvalds (1991) y el proyecto GNU. Ubuntu aparece en 2004
Estructura según 1.4	Núcleo híbrido: base de micronúcleo con subsistemas y capas que corren en modo núcleo	Núcleo monolítico modular: un solo kernel con los servicios básicos, que carga y descarga controladores como módulos mientras está corriendo
Interfaz principal	Gráfica. CMD y PowerShell existen pero son la vía secundaria	Línea de comandos. En esta VM ni siquiera hay entorno gráfico instalado
Procesos medidos	tasklist | find /c /v "" dio 276, o sea 273 procesos	ps aux | wc -l dio 112, o sea 111 procesos
Memoria total	14 141 MB de RAM física, unos 13,8 GB. Quedaban 4 460 MB libres al medir	1,9 GiB asignados a la VM de los 2048 MB configurados. Quedaban 1,6 GiB disponibles y no hay swap
Almacenamiento	Disco físico del Dell Inspiron 16 5645, volumen \Device\HarddiskVolume1	Disco virtual /dev/sda de 20 GB, que es un archivo .vdi. La raíz está en /dev/sda1, 19 GB al 9 %
Usuarios observados	Usuario Lost. En la pestaña Detalles se ven además procesos de SYSTEM, LOCAL SERVICE y NETWORK SERVICE	Usuario estudiante con UID 1000 y GID 1000, miembro de sudo, adm, cdrom, dip, plugdev y users
Arranque	System Boot Time: 30/08/2026 a las 11:53:09 a. m.	Registrado por el kernel. La sesión se abrió el 30/08/2026 a las 21:09 UTC
Dónde corre	Sobre el hardware. Es el anfitrión	Sobre VirtualBox. Es el invitado, y hostnamectl lo confirma con Virtualization: oracle

Preguntas de reflexión

Pregunta 1

En el paso 2 (GUI) y el paso 6 (CMD) contaste procesos de dos formas en Windows. ¿Coincidieron los números? Si no, ¿a qué crees que se debe la diferencia?

No coincidieron, y la diferencia no es pequeña. El Administrador de tareas me daba 208 entradas sumando sus tres bloques (9 aplicaciones, 96 procesos en segundo plano y 103 procesos de Windows). tasklist devolvió 276 líneas, que son 273 procesos una vez que le quito las tres de encabezado. Sesenta y cinco de diferencia. Al principio pensé que había contado mal alguno de los dos.

Lo que pasa es que la pestaña Procesos agrupa. Cuando en la lista aparece «Microsoft Teams (11)», ese número entre paréntesis no es decoración: son once procesos metidos bajo una sola fila. Lo mismo con «Claude (9)», «Steam (8)», «Search (7)» o «VirtualBox Virtual Machine (3)». Windows junta los procesos hijos debajo de la aplicación que los lanzó para que la lista sea legible. tasklist no hace nada de eso, escupe una línea por proceso y ya. La prueba está en la pestaña Detalles del mismo Administrador de tareas: ahí no hay agrupación, cada fila es un proceso, y el número que sale se parece mucho más al de tasklist.

Hay dos causas menores además de esa. Una es que tasklist imprime tres líneas que no son procesos: los títulos de las columnas, la fila de signos igual y una línea en blanco. find las cuenta igual que a las demás porque no sabe qué son, y por eso la guía manda a restar 3.

La otra es que el sistema no se queda quieto entre una medición y la siguiente. Windows arranca y mata procesos todo el tiempo: tareas programadas, comprobaciones de actualización, hosts de servicios que se levantan cuando algo los necesita. Aunque tomé las dos mediciones con segundos de diferencia, es normal que bailen algunas unidades. Y hay algo casi cómico: abrir el CMD y ejecutar el comando crea cuatro procesos nuevos (cmd.exe, conhost.exe, tasklist.exe y find.exe) que terminan contándose a sí mismos.

Me quedo con esto: ninguna de las dos herramientas miente, pero tampoco están contando lo mismo. La ventana te da una vista ya interpretada, pensada para que un usuario entienda qué está pasando; el comando te da el dato crudo. Si no sabes qué está contando la herramienta, el número no significa gran cosa.

Pregunta 2

¿Qué modelo de estructura de S.O. (1.4) corresponde a la VM que acabas de arrancar en VirtualBox? Justifica.

Corresponde al modelo de máquina virtual, uno de los que se ven en el tema 1.4 junto con la estructura monolítica, la de capas, el micronúcleo y el modelo cliente-servidor.

La idea del modelo es que una capa de software, el hipervisor, se pone encima de los recursos físicos y fabrica varias copias virtuales del hardware. Cada copia se comporta como si fuera una computadora completa e independiente, con su procesador, su memoria, sus discos y sus dispositivos. Sobre cada una

se puede instalar un sistema operativo entero, que trabaja sin enterarse de que el hardware que ve es una simulación.

VirtualBox es un hipervisor de tipo 2, o alojado. No se ejecuta directamente sobre el hardware sino como una aplicación más dentro de Windows 11, que hace de anfitrión. Ubuntu 24.04.4 LTS es el invitado y corre sobre el hardware que VirtualBox le presenta.

La justificación con lo que salió en el laboratorio:

Lo primero es que el invitado sabe perfectamente dónde está. hostnamectl responde Virtualization: oracle, Chassis: vm, Icon name: computer-vm, Hardware Vendor: innotek GmbH y Hardware Model: VirtualBox. El propio Linux declara que el hardware es virtual y hasta dice quién se lo provee.

Lo segundo es que el hardware que ve el invitado no tiene nada que ver con el real. Mi máquina es un Dell Inspiron 16 5645 con un Ryzen 7 8840U de 16 procesadores lógicos y 14 141 MB de RAM. Dentro de la VM, nproc devuelve 2 y free -h reporta 1,9 GiB. Exactamente lo que le asigné en el paso 8, ni un poco más. El hipervisor decide cuánto del hardware físico le toca a cada invitado.

Lo tercero es el disco. lsblk muestra /dev/sda de 20 GB y Linux lo trata como trataría a cualquier disco SATA. Visto desde Windows, ese disco es un archivo llamado Lab1-Linux-DISCO-DOCENTE.vdi guardado en una carpeta. El almacenamiento también está virtualizado.

Y por último el aislamiento. Los 111 procesos de la VM no aparecen por ningún lado en la lista de Windows. Desde el anfitrión, la máquina virtual entera se ve como una entrada llamada «VirtualBox Virtual Machine (3)» en el Administrador de tareas. Todo un sistema operativo comprimido en tres procesos, desde afuera.

Ahora, la pregunta se puede leer de dos maneras y creo que vale la pena responder las dos. Si preguntamos por la VM como entorno de ejecución, la respuesta es el modelo de máquina virtual. Pero el sistema que está instalado adentro tiene su propia estructura interna, y esa es de núcleo monolítico modular: los servicios básicos de procesos, memoria, archivos y dispositivos viven todos dentro de un mismo kernel que corre en modo supervisor. Lo de «modular» viene de que puede cargar y descargar controladores mientras está encendido, sin recompilar nada ni reiniciar. Los mensajes de arranque que se ven en la captura del paso 9 son precisamente eso: el kernel cargando módulos y reconociendo dispositivos.

Pregunta 3

Compara la clasificación (1.3) de Windows y de la distribución Linux que usaste: ¿ambos son multiusuario? ¿Ambos son multitarea? Justifica con lo observado en los pasos de conteo de procesos.

Los dos son multiusuario y los dos son multitarea. Pero no lo son de la misma manera, y ahí está lo interesante.

Multitarea

Esta es la parte fácil de demostrar, porque los conteos la dejan servida. Windows tenía 273 procesos activos sin que yo hubiera abierto nada a propósito. La VM de Linux, recién arrancada, sin escritorio y sin una sola aplicación mía, tenía 111.

El detalle es que ninguna de las dos máquinas tiene 273 ni 111 núcleos. El anfitrión tiene 16 procesadores lógicos y la VM tiene 2. Así que esos procesos no están corriendo al mismo tiempo de verdad. El planificador de cada sistema les va repartiendo turnos de CPU en intervalos tan cortos que la sensación desde afuera es de simultaneidad. Eso es multitarea por tiempo compartido, y los dos sistemas la hacen además de forma expropiativa, es decir, el kernel puede quitarle la CPU a un proceso sin pedirle permiso.

La salida de ps aux tiene un detalle que no esperaba. Entre los procesos normales aparecen cosas como [kthreadd], [ksoftirqd/0], [kworker/0:0] o [rcu_preempt], escritas entre corchetes. Son hilos del propio kernel. O sea que no solo las aplicaciones se pelean por el procesador: el núcleo del sistema también tiene sus propias tareas corriendo en paralelo.

Multiusuario

En Linux esto se ve directo, porque viene de UNIX y ahí la idea de varios usuarios compartiendo una máquina es el punto de partida. El comando id me devolvió uid=1000(estudiante) gid=1000(estudiante) y una lista de grupos: adm, cdrom, sudo, dip, plugdev, users. Ese número y esa lista son la base de todo el sistema de permisos. Cada archivo y cada proceso tienen dueño y grupo, y el kernel consulta eso cada vez que alguien intenta hacer algo. La columna USER de ps aux lo confirma: hay procesos de root, los de los servicios y los hilos del kernel, y procesos de estudiante, los de mi sesión. Dos usuarios distintos con procesos vivos al mismo tiempo, en una máquina donde solo hay una persona sentada.

Windows también es multiusuario, aunque cueste más verlo. La pestaña Detalles tiene una columna User name donde aparecen SYSTEM, LOCAL SERVICE, NETWORK SERVICE y mi usuario Lost, cada uno con permisos distintos. tasklist dice lo mismo en su columna Session Name: los servicios corren en la sesión 0 y lo mío en la sesión 1, separados a propósito por seguridad. La diferencia con Linux no es de capacidad técnica sino de para qué se pensó cada uno. Las ediciones de escritorio de Windows están hechas para una persona a la vez y limitan por licencia cuántas sesiones simultáneas puede haber. En Linux que varios usuarios trabajen a la vez es lo normal y no requiere ninguna edición especial.

Resumiendo: los dos cumplen las dos condiciones. Linux te lo muestra en la cara, Windows lo tiene igual pero escondido detrás de una interfaz pensada para que parezca que la máquina es solo tuya.

Pregunta 4

De todos los comandos usados en la Parte A y la Parte B, elige el que más te costó entender y explica, con tus palabras, qué información entrega y para qué le serviría a alguien que administra un servidor.

El que más me costó fue tasklist | find /c /v "". Puntualmente el /v "" del final.

tasklist se entiende solo, lista procesos. El pipe también, una vez que te lo explican: lo que un comando iba a escribir en pantalla se lo pasa a otro como entrada. Lo que me trabó fueron las opciones de find. La

/c dice «cuenta en vez de mostrar», hasta ahí bien. La /v invierte la búsqueda: en vez de quedarse con las líneas que contienen el texto, se queda con las que no lo contienen. Y como el texto que se busca es la cadena vacía, que técnicamente está dentro de cualquier línea, pedirle lo contrario equivale a decir «cuéntame todas las líneas que tengan algo escrito».

Me parece un rodeo bastante feo. Estás usando un buscador de texto para hacer algo que no tiene nada que ver con buscar texto. Cuando después vi el equivalente en Linux, ps aux | wc -l, me dio hasta rabia: wc -l existe para contar líneas y ya está, no hay truco.

Lo que entrega el comando es un solo número: cuántos procesos hay vivos en el sistema en ese momento, contando las tres líneas de encabezado que hay que restarle.

A alguien que administra un servidor le sirve por varias razones. La primera es tener una referencia. Si un servidor anda siempre por los 120 procesos y un día marca 400, algo cambió. Puede ser una aplicación con fuga que abre procesos y no los cierra, algo bifurcándose sin control, o directamente alguien haciendo cosas que no debería. El número no te dice qué pasó, pero te avisa que hay que ir a mirar.

La segunda es que un número pesa nada. Lo puedes escribir en un log cada minuto y graficarlo durante meses. Guardar la salida completa de tasklist con esa frecuencia sería inmanejable.

La tercera es que al ser un valor numérico se puede comparar dentro de un script. Si pasa de cierto umbral, que mande un correo, que reinicie un servicio, que escriba en algún lado. Ahí es donde deja de ser un comando suelto y se vuelve monitoreo automático. También sirve como comprobación rápida después de tocar algo: aplicas una actualización, reinicias un servicio, y comparas el conteo de antes con el de después.

Y hay una idea de fondo que me quedó dando vueltas. Nadie programó una herramienta llamada «contar procesos». Ese resultado salió de pegar dos utilidades chiquitas que hacen otra cosa. Programas pequeños, cada uno bueno en lo suyo, que se encadenan para resolver algo que ninguno de los dos resolvía solo. Es la base de trabajar por consola, y funciona igual en Windows que en Linux.

Pregunta 5

¿Por qué crees que en Linux gran parte de la administración del sistema se hace por línea de comandos, mientras que en Windows la mayoría de los usuarios nunca abre una terminal?

Creo que no es una diferencia técnica. Es que cada sistema se diseñó pensando en gente distinta, y esa decisión inicial se quedó pegada.

De dónde viene cada uno

Linux desciende de UNIX, que nació en 1969 en los laboratorios Bell para computadoras que muchos usuarios compartían a través de terminales de texto. En ese momento las interfaces gráficas no eran una opción, así que todas las herramientas de administración se escribieron como programas de texto que leen de la entrada estándar y escriben en la salida estándar. Cuando años después llegaron los escritorios gráficos al mundo UNIX, no reemplazaron nada: se pusieron encima como una capa opcional. La VM de este laboratorio es la prueba, funciona perfecto sin entorno gráfico instalado.

Windows hizo el camino contrario. Se pensó desde el principio para llevar la computación a gente sin formación técnica, en máquinas personales de un solo dueño. La interfaz gráfica no es una capa que se pueda quitar: es el producto. CMD sobrevive como herencia de MS-DOS y como salida de emergencia. PowerShell llegó muchísimo después, y llegó porque Microsoft se dio cuenta de que administrar cientos de máquinas a clics no escala.

Y por qué se quedó así

Buena parte de los sistemas Linux del mundo son servidores a los que se entra por SSH, sin monitor ni teclado conectados. Cargarles un escritorio sería gastar memoria y CPU en algo que nadie va a mirar, y de paso darle más superficie a quien quiera atacarlos.

Además, un comando es texto. Se guarda, se comenta, se manda por correo, se pone en un manual y se repite igualito en mil servidores. Una secuencia de clics no se guarda ni se repite sola. Por eso todo lo que hoy se llama automatización, sean scripts, Ansible o contenedores, se escribe en comandos y no en capturas de pantalla. Lo mismo vale para dejar rastro: en una auditoría el historial de comandos es evidencia, y acordarse de qué botones se apretaron no lo es.

Está también lo que vimos en la pregunta anterior. Los comandos se combinan entre sí, las ventanas no. Cada ventana hace lo que su programador previó y nada más. Y en Linux la información del sistema está expuesta como archivos de texto que se pueden leer con las herramientas de siempre, como pasó con /etc/os-release en el paso 11. En Windows buena parte de esa información vive en el Registro y en objetos WMI, que son estructuras binarias donde la ventana gráfica es la forma natural de consultar.

No creo que una forma sea mejor que la otra. Optimizan cosas distintas: Windows optimiza que una persona frente a una pantalla la pase bien, Linux optimiza control, repetibilidad y administración a distancia. Y de hecho los dos mundos se están acercando. Windows tiene PowerShell, Windows Server se puede instalar sin interfaz gráfica y ahora trae WSL para correr Linux adentro. Linux tiene escritorios completos y herramientas de administración visuales para quien las quiera. Lo que queda de la diferencia es más cultural que otra cosa.

Anexos

Anexo A. Comandos ejecutados

Paso	Dónde	Comando	Para qué
1	Windows, ventanas	winver	Edición, versión y build del sistema (1.1)
2	Windows, ventanas	Administrador de tareas	Ver la multitarea con los ojos (1.3 y 1.5)
3	Windows, CMD	cmd	Entrar a la línea de comandos (1.7)
4	Windows, CMD	systeminfo | more	Informe del equipo con la hora de arranque (1.6)
4	Windows, CMD	systeminfo | findstr ...	Recortar la salida a lo que pide la guía
5	Windows, CMD	tasklist | more	Procesos con PID y memoria (1.5)
6	Windows, CMD	tasklist | find /c /v ""	Contar procesos usando una tubería
7	VirtualBox	Botón Nueva, disco .vdi existente	Modelo de máquina virtual (1.4.3)
8	VirtualBox	Configuración, Sistema	Repartir recursos del anfitrión al invitado
9	VirtualBox	Botón Iniciar e inicio de sesión	Arranque del sistema invitado (1.6)
10	Linux, CLI	uname -a	Kernel y arquitectura (1.1 y 1.2)
11	Linux, CLI	cat /etc/os-release	Distribución, leída como archivo de texto
11	Linux, CLI	hostnamectl	Sistema, kernel y confirmación de que es una VM
12	Linux, CLI	whoami e id	Usuario, UID y grupos (1.3 y 1.5)
13	Linux, CLI	ps aux y ps aux | wc -l	Gestión de procesos y multitarea (1.5)
14	Linux, CLI	free -h	Memoria, adelanto del Módulo II
14	Linux, CLI	df -h	Sistema de archivos, adelanto del Módulo III
14	Linux, CLI	nproc y lsblk	CPU asignadas y discos detectados

Anexo B. Archivos que acompañan al informe

Carpeta o archivo	Qué contiene
capturas	Las capturas de pantalla de cada comprobación. Empiezan con A_ las de la Parte A y con B_ las de la Parte B
salidas\systeminfo.txt	La salida completa de systeminfo
salidas\tasklist.txt	El listado completo de procesos de Windows
salidas\conteo_procesos.txt	El resultado del conteo por tubería

Anexo C. Configuración final de la máquina virtual

Para dejar constancia de cómo quedó la VM, la misma información de las capturas pero consultada por comandos:

```
C:\> VBoxManage showvminfo "Lab1-Linux" --machinereadable
ostype="Ubuntu (64-bit)"
memory=2048
cpus=2
VMState="running"
"SATA-0-0"="C:\Users\Lost\VMs\Lab1-Linux\Lab1-Linux-DISCO-DOCENTE.vdi"
"IDE-0-0"="C:\Users\Lost\VMs\seed-cidata.iso"
```

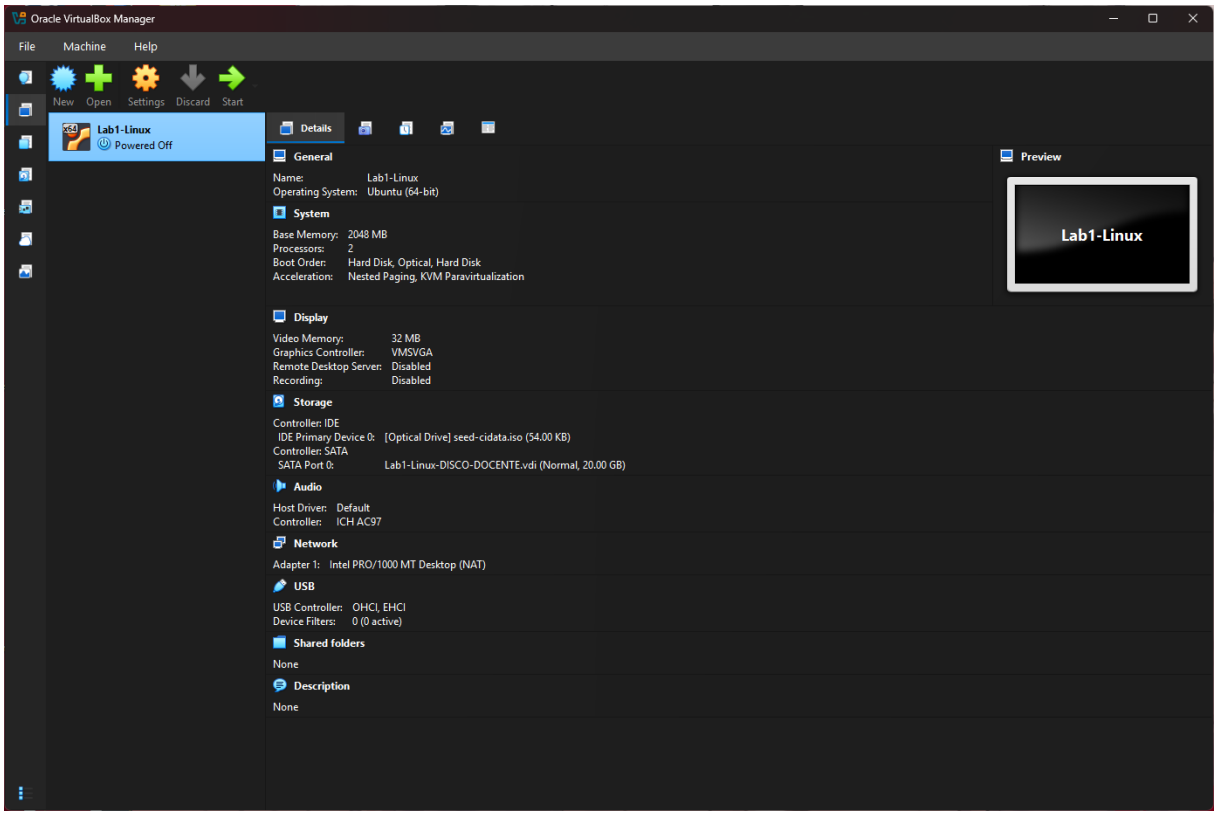


Figura 30. Estado final de Lab1-Linux en VirtualBox Manager, con los recursos asignados y el disco conectado.